UNIVERZITA KONŠTANTÍNA FILOZOFA V NITRE

FAKULTA PRÍRODNÝCH VIED A INFORMATIKY

LEXIKÁLNA ANALÝZA SENTIMENTU APLIKOVATEĽNÁ NA SLOVENSKÉ TEXTY

DIPLOMOVÁ PRÁCA

2026 **Bc. Matej Lužák**

UNIVERZITA KONŠTANTÍNA FILOZOFA V NITRE

FAKULTA PRÍRODNÝCH VIED A INFORMATIKY

LEXIKÁLNA ANALÝZA SENTIMENTU APLIKOVATEĽNÁ NA SLOVENSKÉ TEXTY

DIPLOMOVÁ PRÁCA

Študijný odbor:	18. Informatika
Študijný program:	Aplikovaná informatika
Školiace pracovisko:	Katedra informatiky
Školiteľ:	doc. PaedDr. Martin Magdin, PhD.

Nitra 2026

Bc. Matej Lužák

309245

Univerzita Konštantína Filozofa v Nitre
Fakulta prírodných vied a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Bc. Matej Lužák
Študijný program: aplikovaná informatika (Jednoodborové štúdium, magisterský II. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: Diplomová práca
Jazyk záverečnej práce: slovenský
Sekundárny jazyk: anglický

Názov: Lexikálna analýza sentimentu aplikovateľná na slovenské texty

Anotácia: V súčasnosti sa ako primárny komunikačný prostriedok používajú jednoduché texty (tweety) na implicitné alebo explicitné vyjadrenie emócií. Detekcia sentimentu z textu sa stala dôležitou a rozširujúcou sa oblasťou výskumu pre jasnejšie pochopenie skutočných pocitov ľudí. Väčšina modelov reprezentácie slov, ako napríklad Word2Vec alebo GloVe, premieta slová vo vektorovom priestore tak, že ak majú slová podobný kontext, ich reprezentácie sú tiež veľmi podobné. Tento postup však je aplikovateľný najmä iba pre anglické texty. V prípade slovenského jazyka chýba komplexný vektorový slovník slov, čo výrazne obmedzuje použitie týchto metód práve v oblastiach, ako je detekcia sentimentu. Nedávno výskumníci navrhli niektoré metódy založené na pridávaní sentimentálnych informácií k pôvodným slovným vektorom. Tie zlepšili vektorovú reprezentáciu slov a dosiahli lepšie výsledky v oblasti klasifikácie.

Cieľ práce: Cieľom bude návrh slovníka na princípe lexikálnej analýzy, ako aj vývoj riešenia pre automatické rozpoznávania a klasifikovania sentimentu na základe tohto spôsobu analýzy.

Charakter práce:
aplikačný – popis riešeného problému, návrh systémov a hardvérových riešení, metodika tvorby a implementácie, testovanie a porovnanie simulovaného a fyzického riešenia

Predmetové prerekvizity:
Vývoj mobilných aplikácií (2., bc)
Úvod do softvérového inžinierstva (3., bc.)
Úvod do umelej inteligencie (3., bc.)

Najdôležitejšie kompetentnosti získané spracovaním témy:
formulácia požiadaviek na vývoj softvéru a kontrola požadovaných funkčností a funkcionalít vyvíjaného softvéru
sledovanie aktuálnych trendov a inovácií vo vývoji softvérových riešení
navrhovanie vhodných softvérových technológií pre konkrétne zadanie, vrátane návrhu riešení na optimalizáciu a zvyšovanie efektívnosti prostriedkov výpočtovej techniky

implementácia vhodných metód a techník na vývoj softvéru a testovanie vyvíjaných softvérových riešení
aplikácia metód a techník vývoja a integrácie softvérových komponentov do rozsiahlejšieho celku (výber, vyhodnotenie a integráciu rôznych softvérových komponentov a modulov do aplikačného celku)

Školiteľ: doc. PaedDr. Martin Magdin, Ph.D.
Oponent: prof. RNDr. Daša Munková, PhD.
Katedra: KI - Katedra informatiky

Spôsob sprístupnenia elektronickej verzie práce:
bez obmedzenia

Dátum zadania: 07.10.2024

Dátum schválenia: 20.12.2024

prof. RNDr. Michal Munk, PhD., v. r.
osoba zodpovedná za realizáciu študijného programu

POĎAKOVANIE

Touto formou by som chcel poďakovať vedúcemu diplomovej práce pánovi doc. PaedDr. Martin Magdin, PhD. za odborné vedenie, poradenstvo a poskytnutie odborných materiálov počas realizácie tejto záverečnej práce.

ABSTRAKT

LUŽÁK, Matej: Lexikálna analýza sentimentu aplikovateľná na slovenské texty [Diplomová Práca].

Univerzita Konštantína Filozofa v Nitre, Fakulta prírodných vied a informatiky.

Školiteľ: doc. PaedDr. Martin Magdin, PhD.

Stupeň odbornej klasifikácie: Magister odboru Aplikovaná informatika. Nitra: FPVaI, 2026, 65 s.

Analýza sentimentu je veľmi aktuálna a obsiahla téma. Správna identifikácia polarity sentimentu v rôznych doménach je veľmi dôležitá nielen pre identifikáciu zámeru danej osoby, ale aj pre predpoveď budúceho správania. V tejto práci sa konkrétne zameriavame na lexikálny prístup analýzy sentimentu, pričom predstavujeme nielen spoľahlivý zdroj pre sentimentálnu analýzu vo forme slovníka sentimentu, ale aj lexikálny Support Vector Machine model schopný klasifikovať text do dvoch tried na základe polarity sentimentu. Okrem spomínaných výsledkov sme porovnávali viacero metód klasifikácie sentimentu. Vytvorili sme hybridný model RoBERTa, ktorý sme porovnali so základným modelom RoBERTa. Práca sme realizovali prostredníctvom rozhrania Google Colab, pričom na samotné zostrojenie slovníka sentimentu sme použili litovský SlovakWordNet a SentiWordNet 3.0. Trénovanie a testovanie viacerých fáz praktickej časti tejto práce prebiehali prostredníctvom sentimentom ohodnotených recenzií v slovenskom jazyku SlovakSA. Naše testovanie ukázalo, že transformerové modely prekonávajú všetky dostupné metódy analýzy sentimentu. Napriek tomu nami prezentovaný slovník sentimentu, ako aj Support Vector Machine model, predstavujú vhodný baseline na klasifikáciu sentimentu v slovenskom jazyku.

Kľúčové slová: Spracovanie prirodzeného jazyka. Analýza sentimentu. Lexikálna analýza sentimentu. Slovenská analýza sentimentu. RoBERTa.

ABSTRACT

LUŽÁK, Matej: Lexical Sentiment Analysis Applicable to Slovak Texts [master's thesis].

Constantine the Philosopher University in Nitra, Faculty of Natural Sciences and Information Technology.

Tutor: doc. PaedDr. Martin Magdin, PhD.

Degree of professional qualification: Master's degree in applied informatics. Nitra: FPVaI, 2026, 65 p.

Sentiment analysis is a highly relevant and extensive topic. The correct identification of sentiment polarity across various domains is crucial not only for determining an individual's intent but also for predicting future behavior. In this work, we specifically focus on the lexicon-based approach to sentiment analysis, presenting not only a reliable resource for sentiment analysis in the form of a sentiment lexicon, but also a lexicon-based Support Vector Machine model capable of classifying text into two classes based on sentiment polarity. In addition to the results, we compared multiple sentiment classification methods. We developed a hybrid RoBERTa model, which we compared with a baseline RoBERTa model. The work was carried out using the Google Colab environment, and for the construction of the sentiment lexicon, we utilized the Lithuanian SlovakWordNet and SentiWordNet 3.0. The training and evaluation of multiple phases of the practical part of this work were conducted using sentiment-annotated reviews in the Slovak language from the SlovakSA dataset. Our evaluation showed that transformer-based models outperform all other available sentiment analysis methods. Nevertheless, the presented sentiment lexicon, as well as Support Vector Machine model represent a suitable baseline for sentiment classification in the Slovak language.

Keywords: Natural Language Processing. Sentiment analysis. Lexicon-based sentiment analysis. Slovak sentiment analysis. RoBERTa.

OBSAH

1 Analýza súčasného stavu 10

1.1 Spracovanie prirodzeného jazyka 10

1.2 História a vývoj NLP 10

1.3 Tokenizácia a predspracovanie textu 10

1.3.1 Lematizácia a Stemming 11

1.3.2 Odstraňovanie stop slov 11

1.3.3 Part of Speech Tagging 12

1.4 Reprezentácia slov vo vektorovom priestore 12

1.4.1 One Hot Encoding 13

1.4.2 Vector space model 14

1.4.3 Word embedding 14

1.4.4 Bag of Words 15

1.4.5 Continuous Bag of Words 16

1.4.6 Skip-Gram Model 17

1.4.7 GloVe 17

1.4.8 FastText 17

1.4.9 Word2Vec 18

1.5 Transformery a moderné jazykové modely 18

1.5.1 BERT a jeho varianty 19

1.5.2 RoBERTa 20

1.5.3 DistilBERT 20

1.6 Analýza Sentimentu 20

1.6.1 Lexikálna analýza sentimentu 21

1.6.2 Sentiwordnet 21

1.7 Strojové učenie a analýza sentimentu 22

1.8 Porovnanie online nástrojov analýzy sentimentu 23

1.8.1 IBM Watson NLP 23

1.8.2 Google Cloud NLP API 24

1.8.3 AWS Comprehend 24

1.8.4 NLP Cloud 25

2 Ciele záverečnej práce 26

3 Návrh a Metodika 27

3.1 Návrh 27

3.2 Metodika Vývoja 27

3.2.1 Porozumenie dátam 28
3.2.2 Príprava dát 28
3.2.3 Modelovanie 29
3.2.4 Testovanie 30
4 Výsledky 31
4.1 Príprava dát 31
4.1.2 Mapovanie synsetov SlovakWordNet na SentiWordNet 3.0 34
4.1.3 Preklad synsetov SentiWordNet 3.0 35
4.1.4 Testovanie lexikónu bez lematizácie 40
4.1.5 Testovanie lexikónu s lematizáciou 43
4.2 Modelovanie 45
4.2.1 Základný model RoBERTa 45
4.2.2 Hybridný model RoBERTa 48
4.2.3 Lexikálna klasifikácia prostredníctvom Support Vector Machine 51
4.2.4 Automatická klasifikácia textu 53
4.3 Testovanie 54
4.4 Diskusia 56
Záver 58
Zoznam bibliografických odkazov 60
Zoznam príloh 64

ZOZNAM ILUSTRÁCII

Obrázok 1: Ukážka metódy One Hot Encoding. 13
Obrázok 2: Ukážka vektorového priestoru. 15
Obrázok 3: Ukážka reprezentácie slov Bag Of Words. 16
Obrázok 4: Príklad neurónovej siete Word2Vec. 18
Obrázok 5: Architektúra transformeru. 19
Obrázok 6: Príklad tabuľky SentiWordNet. 29
Obrázok 7: Príklad tabuľky SlovakSA. 29
Obrázok 8: Výsledná tabuľka SentiWordNet. 32
Obrázok 9: Výsledná tabuľka SlovakWordNet. 34
Obrázok 10: Výsledné metriky testovania bez lematizácie. 42
Obrázok 11: Výsledné metriky testovania po lematizácii. 44
Obrázok 12: Výsledný DatasetDict SlovakSA. 45
Obrázok 13: Výsledné metriky základného modelu RoBERTa. 48
Obrázok 14: Ukážka datasetu train_lem s lexikálnymi features. 49
Obrázok 15: Výsledné metriky hybridného modelu RoBERTa. 50
Obrázok 16: Výsledné metriky SVM. 52
Obrázok 17: Príklad použitia triedy SlovakSentimentModel. 54

ZOZNAM TABULIEK

Tabuľka 1: Porovnanie online nástrojov analýzy sentimentu. 25
Tabuľka 2: Porovnanie prístupov klasifikácie sentimentu. 55

ÚVOD

Analýza sentimentu je v dnešnom svete veľmi dôležitá nielen na rozpoznanie zámeru autora, ale aj na predvídanie jeho správania v budúcnosti. S neustále narastajúcou popularitou veľkých jazykových modelov ako sú GPT, Claude a Gemini je extrahovanie informácii z textu vrátane sentimentálnej polarity veľmi aktuálna téma. Prostredníctvom strojového učenia a transformerových modelov vieme analýzu sentimentu plne automatizovať, čo je v neustále narastajúcom množstve dostupných informácii dôležité. V tejto práci sa budeme venovať primárne lexikálnej analýze sentimentu, pričom porovnáme rôzne metódy analýzy sentimentu.

V teoretickej časti práce sa najskôr oboznámime z rôznymi metódami spracovania prirodzeného jazyka od tokenizácie textu, až po analýzu sentimentu prostredníctvom rôznych metód strojového učenia. Rozoberieme slovníky sentimentu a taktiež poukážeme na dostupné nástroje analýzy sentimentu ako sú IBM Cloud, Amazon Comprehend, a podobne.

Cieľom tejto práce je vytvoriť slovenský slovník sentimentu a natrénovať model strojového učenia na binárnu klasifikáciu sentimentu v slovenskom jazyku. Kombináciou litovského SlovakWordNetu a strojového prekladu synsetov SentiWordNetu vytvoríme slovník sentimentu pre slovenský jazyk. Vytvoríme hybridný transformerový model RoBERTa s použitím údajov extrahovaných z vytvoreného slovníka, pričom porovnáme jeho výkon so základným modelom RoBERTa. Následne vytvoríme a natrénujeme lexikálny Support Vector Machine, ktorý bude schopný klasifikovať text do dvoch tried, a to: pozitívny a negatívny. Porovnáme testované metódy analýzy sentimentu a vyhodnotíme spoľahlivosť vytvoreného slovníka sentimentu, ako aj natrénovaného Support Vector Machine modelu.

1 ANALÝZA SÚČASNÉHO STAVU

Skôr, ako sa dostaneme k nami navrhovanému riešeniu vektorovej analýzy sentimentu pre slovenské texty, je potrebné zadefinovať základné pojmy. V úvodnej kapitole sa budeme bližšie zaoberať analýzou súčasného stavu. Pozrieme sa na vývoj nástrojov spracovania prirodzeného jazyka, pričom poukážeme a porovnáme rôzne navrhnuté riešenia analýzy sentimentu v texte. Budeme sa primárne zaoberať spomenutou analýzou sentimentu, pričom sa sústredíme aj na rôzne formy vektorovej reprezentácie textu.

1.1 SPRACOVANIE PRIRODZENÉHO JAZYKA

Spracovanie prirodzeného jazyka (*Natural Language Processing*) predstavuje skupinu metód, ktoré umožňujú počítačom interpretovať a porozumieť ľudskému jazyku. Spracovanie prirodzeného jazyka (ďalej už len NLP) je v poslednom desaťročí súčasťou nášho každodenného života, pričom klasifikácia písaného textu nám poskytuje efektívny spôsob na získavanie a zdieľanie informácií. NLP je sústredené na dizajn a analýzu výpočtových algoritmov, ktorých cieľom je poskytnúť nové výpočtové možnosti ohľadom ľudského jazyka, ako napríklad extrahovanie informácii z textu, prekladmi rôznych jazykov, odpovedanie na otázky, a podobne. (Jacob Eisenstein, 2019).

1.2 HISTÓRIA A VÝVOJ NLP

Spracovanie prirodzeného jazyka má bohatú históriu, ktorú autori (Supriyono et al., 2024) popisujú od 50. rokov 20. storočia, kedy vznikli prvé pokusy v oblasti strojového prekladu a počítačovej lingvistiky. Rozvoj bol zo začiatku sústredený na systémy založené na pravidlách zamerané na gramatiku a syntax. V 80. a 90. rokoch prišli štatistické metódy, ktoré využívali korpusy textových dát a pravdepodobnostné modely. Prelom 21. storočia znamenal vývoj strojového učenia, ktorý postúpil až do hlbokého učenia a modelov založených na transformátoroch.

1.3 TOKENIZÁCIA A PREDSPRACOVANIE TEXTU

Štúdie spracovania prirodzeného jazyka sa zväčša sústreďujú na analýzu alebo generovanie, pričom základné jednotky (slová) sa považujú za samozrejmé. Avšak bez

jasného oddelenia a definície týchto základných jednotiek nie je možné vykonať žiadnu analýzu ani generovanie. To znamená, že prvým krokom v akejkoľvek NLP úlohe je identifikácia tokenov, ktoré nie je potrebné ďalej rozkladať v ďalšom spracovaní. Entitné slovo predstavuje jeden z najbežnejších a zároveň najzákladnejších tokenov pre NLP (Webster & Kit, 1992).

Existuje niekoľko metód na identifikáciu tokenov, ako napríklad:

- Lexikálny Information Retrieval,
- Table-look-up Matching,
- Generalizovaný Table-look-up.

Pokiaľ ide o Lexikálny IR (*Information Retrieval*), entita môže byť chápaná ako súčasť siete vzťahov. Jednotlivé entity v lexikóne zahŕňajú slová a ich časti, modely a kontexty použitia. Ide o lexémy, ktoré sú systematicky spojené medzi sebou v dvojrozmernom priestore. *Table-look-up matching* predpokladá, že existuje dopredu vytvorený zoznam tokenov. Najskôr sa tokenizuje každé jednotlivé slovo a následne sa porovnávajú ostatné, aby sa zistilo, či sa medzi nimi nachádzajú zložené tokeny. Generalizovaná *Table-look-up* metóda predstavuje upravenú *Table-look-up* metódu na spracovanie idiómov a ustálených výrazov (Webster & Kit, 1992).

1.3.1 LEMATIZÁCIA A STEMMING

V dokumentoch sa používajú nielen rôzne formy slov, ale aj rôzne odvodené slová od rovnakého slovného základu. Cieľom *stemmingu* a lematizácie je zredukovať skloňované a odvodené tvary slova na základnú formu. Napríklad z výrazu „*the boy's cars are different colors*“ sa stane „*the boy car be differ color*“. Stemming zvyčajne predstavuje hrubý heuristický proces, ktorý odstraňuje koncovky slov v nádeji, že sa tento cieľ podarí dosiahnuť správne. Lematizácia sa sústreďuje na správny postup s použitím slovnej zásoby a morfologickej analýzy slov, pričom jej cieľom je odstrániť koncovky a vrátiť základnú formu slova, ktorá sa nazýva lema. Napríklad pri slove *saw stemming* môže vrátiť len *s*, pričom lematizácia sa pokúsi vrátiť buď *see,* alebo *saw*, na základe toho, či ide o sloveso alebo podstatné meno (Vechtomova, 2009).

1.3.2 ODSTRAŇOVANIE STOP SLOV

Stop slová v NLP predstavujú bežné slová, ktoré sa často vynechávajú pri spracovaní textu, pretože sa vyskytujú príliš často a málo prispievajú k významu.

Príklady zahŕňajú členy (a, the, an), predložky ako (in, on, at), spojky a zámená. Tieto slová sa filtrujú, aby sa znížil celkový šum textu. Primárne ide o zvýšenie efektivity a presnosti pri úlohách, ako sú vyhľadávanie, modelovanie alebo analýza sentimentu. Existujú však výnimky, pretože v niektorých kontextoch majú stop slová kritický význam. Napríklad negačné frázy ako „*not good*“ strácajú svoj význam po odstránení negácie „*not*“ (*What Are Stop Words in NLP?*, n.d.).

1.3.3 PART OF SPEECH TAGGING

POS tagging alebo označovanie slovných druhov je proces, kde každému slovu vo vete je priradená gramatická kategória. Ako napríklad podstatné meno, sloveso, prídavné meno, a podobne. Tento proces pomáha počítačom pochopiť štruktúru a význam viet identifikovaním vzájomných vzťahov medzi slovami. Existuje niekoľko metód na *POS tagging*, ako (*POS(Parts-Of-Speech) Tagging in NLP*, 2025):

- Rule-Based Tagging,
- Transformation-Based Tagging,
- štatistický POS Tagging.

POS tagging hrá dôležitú úlohu v rôznych aplikáciách NLP, vrátane strojového prekladu, analýzy sentimentu (*POS(Parts-Of-Speech) Tagging in NLP*, 2025).

1.4 REPREZENTÁCIA SLOV VO VEKTOROVOM PRIESTORE

Ako uvádzajú autori (Badman & Kosinski, 2024), dáta predstavujú kolekciu faktov, čísiel, slov alebo iných užitočných informácií. Organizácie zbierajú dáta z rôznych zdrojov v rôznych formátoch. Tieto dáta sa najčastejšie delia na:

- kvalitatívne,
- kvantitatívne,
- štruktúrované,
- neštruktúrované,
- metadáta,
- veľké dáta.

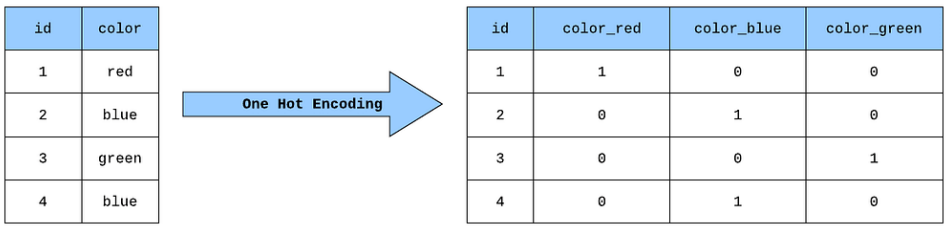
Medzi neštruktúrované dáta patria komplexnejšie formáty ako text, dokumenty, videá a obrázky (Badman & Kosinski, 2024).

Textové dáta sa dajú v rôznych fázach reprezentovať vektormi. Jeden vektor môže reprezentovať dokument, paragraf, slovo alebo aj jeden znak. Jedným z

najpoužívanejších modelov na vektorovú reprezentáciu dokumentov je *Information Retrieval* (*IR*). Metódy zberu informácií sa pokúšajú nájsť najrelevantnejší dokument pre daný dotaz. Najjednoduchší hodnotiaci model je Boolean model, ktorý je založený na výskyte daných výrazov v dokumente, pričom dokumenty, ktoré majú najväčšiu zhodu, sú postavené na vrchol rebríčka (Grzegorczyk Advisor et al., 2019).

1.4.1 ONE HOT ENCODING

Ako popisuje autor (Egger, 2022), najjednoduchší spôsob reprezentácie slov predstavuje takzvaný *One Hot Encoding*, kde sú kategorické dáta reprezentované ako binárne vektory. V prvom kroku sú všetky dáta (slová) zmapované na binárne hodnoty a následne môže byť každá hodnota reprezentovaná ako binárny vektor vo forme núl a jednotiek, ako môžeme vidieť na obrázku (*Obrázok 1*).



Obrázok 1: Ukážka metódy One Hot Encoding.[1]

Napriek tomu, že je táto metóda veľmi jednoduchá na implementáciu, má niekoľko vážnych nedostatkov. Keďže touto metódou dokážeme identifikovať len samotné slová, kontext daných slov je úplne stratený. To znamená, že počítač nedokáže určiť akýkoľvek vzťah so susednými slovami, vetami alebo paragrafmi. Táto metóda taktiež vyžaduje veľa pamäte na výpočet. Z toho vyplýva, že potrebujeme komplexnejšie metódy na zachovanie kontextu daných slov (Egger, 2022).

Ďalej sa pozrieme na niektoré z najpoužívanejších modelov *Information Retrieval*, a to:

- Vector Space Model,
- vkladanie slov (*word embedding*),
- Bag of Words model.

[1] Zdroj Obrázok 1: https://miro.medium.com/v2/resize:fit:1100/format:webp/1*ggtP4a5YaRx6l09KQaYOnw.png

1.4.2 VECTOR SPACE MODEL

Indexovanie kľúčových slov je najlepší spôsob na spracovanie veľkého množstva textu. Indexy sa môžu pohybovať od jednotlivých slov po komplexnejšie frázy, pričom bežné slová ako *the*, *and*, *to* a podobné, sú ignorované, pretože nemajú veľkú výpovednú hodnotu a často sa opakujú. Štatistický *Vector Space Model* reprezentuje dokument vektormi kľúčových slov, spolu s priradenými váhami, ktoré reprezentujú dôležitosť daného kľúčového slova v dokumente. Váhu kľúčového slova vieme určiť rôznymi spôsobmi. Jedným z nich je metóda *Inverse Document Frequency* (IDF), kde váha daného slova je určená dvoma faktormi, a to:

- ako často sa výraz vyskytuje v dokumente (frekvencia výrazu),
- ako často sa výraz vyskytuje v kolekcii dokumentov (dokumentová frekvencia).

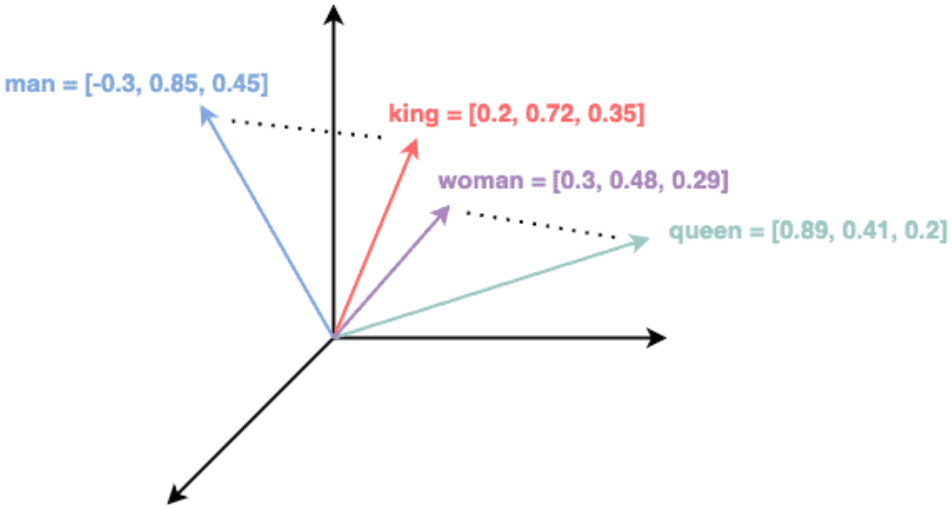
To znamená, že váha výrazu je určená ako $w_{i,j} = tf_{i,j} \times idf_j = tf_{i,j} \times \log N/df_j$, kde N reprezentuje počet dokumentov (Lee & Kong, 1997).

1.4.3 WORD EMBEDDING

Reprezentovanie slov a dokumentov je veľkou súčasťou spracovania prirodzeného jazyka (*NLP*). Reprezentácia slov vektormi je veľmi užitočná, pretože umožňuje použitie viacerých operácií (sčítanie, odčítanie, porovnávanie vzdialeností) a umožňuje využitie algoritmov strojového učenia. *Vector Space Model* je najúspešnejší model, ktorý reprezentuje slová a dokumenty vektormi (Almeida & Xexéo, 2019).

Vkladanie slov je projekcia slov v priestore, pričom sú zachované sémantické a syntaktické podobnosti medzi nimi. Vytvára sa n-rozmerný vektorový priestor, ktorý opisuje podobné slová (*hotel*, *hostel*) alebo sémanticky súvisiace slová (*restaurant*, *food*). Na základe trénovacieho korpusu sú podobné a sémanticky súvisiace slová umiestnené blízko seba vo vektorovom priestore (Egger, 2022).

Názornú ukážku vektorového priestoru môžeme vidieť znázornenú na obrázku (*Obrázok 2*).



Obrázok 2: Ukážka vektorového priestoru.[2]

Existuje viacero spôsobov vkladania slov, ako napríklad:

- Bag of Words,
- Common Bag of Words (CBOW),
- Skip-gram,
- GloVe,
- FastText,
- Word2Vec.

Tieto metódy sa odlišujú v architektúre a type dát, ktorými sú trénované (Ghannay et al., 2016).

1.4.4 BAG OF WORDS

Bag of Words (*BoW*) môžeme chápať ako výber variabilnej podmnožiny, výber atribútov alebo výber premenných. Jedná sa o model, ktorý sa využíva ako algoritmus na extrahovanie vlastností, klasifikáciu dokumentov a obrázkov. Pri klasifikácii textu, *BoW* predstavuje vektor početností výskytu jednotlivých slov, ktorý sa taktiež nazýva aj histogramom dokumentu. Inými slovami, metódu *BoW* môžeme definovať ako histogramovú reprezentáciu založenú na nezávislých vlastnostiach (Qader et al., 2019). Model *BoW* sa dá využiť na niekoľko účelov, medzi ktoré patria:

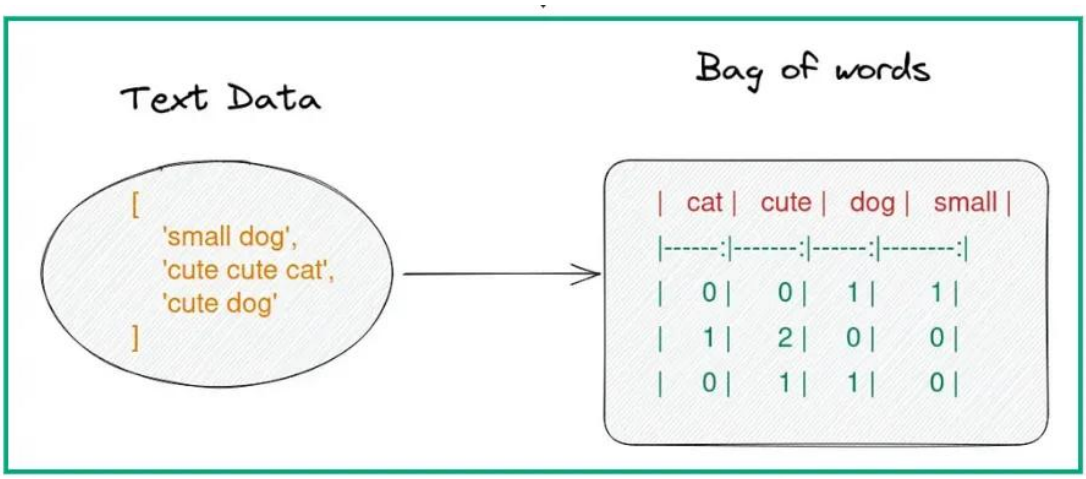
- klasifikácia textu a dokumentov,

[2] Zdroj Obrázok 2: https://www.baeldung.com/wp-content/uploads/sites/4/2023/03/word_embedding.drawio.png

- rozpoznávanie a klasifikácia obrázkov.

Bag of Words model využíva, rovnako ako *Vector Space* model, inverznú dokumentovú frekvenciu (*IDF*) na zdôraznenie dôležitých termínov, čím sa síce zachytila ich dôležitosť, ale stratili sa dôležité informácie o poradí slov. To znamená, že vety ako *dog bites man* a *man bites dog*, sa stali identickými (Das Debasish, 2025).

Na obrázku (*Obrázok 3*) môžeme vidieť príklad transformácie textových dát na vektorovú reprezentáciu *Bag of Words*.



Obrázok 3: Ukážka reprezentácie slov Bag Of Words.[3]

1.4.5 CONTINUOUS BAG OF WORDS

CBOW je algoritmus založený na neurónovej sieti, ktorý predpovedá cieľové slovo na základe kontextu, podobne ako aj *Skip-Gram* model. Ide o učenie bez učiteľa a často sa používa na predbežné trénovanie vkladania slov (*Continuous Bag of Words (CBOW) in NLP - GeeksforGeeks*, 2025).

Model *BoW* reprezentuje text ako súbor slov a ich frekvenciu v danom dokumente alebo korpuse. Neberie do úvahy poradie ani kontext, v ktorom sa slová vyskytujú, a preto nemusí zachytiť celý význam textu. Na rozdiel od *BoW* je model *CBOW* založený na neurónovej sieti, ktorý zachytáva kontext slov. Zohľadnením okolitých slov môže model *CBOW* lepšie zachytiť význam slova v danom kontexte (*Continuous Bag of Words (CBOW) in NLP - GeeksforGeeks*, 2025).

[3] Zdroj Obrázok 3: https://miro.medium.com/v2/resize:fit:1100/format:webp/0*eMSynnbXMnmaGNiR.png

1.4.6 SKIP-GRAM MODEL

Cieľom *Skip-Gram* modelu je nájsť reprezentácie slov, ktoré sú užitočné na predikciu susedných slov vo vete alebo v dokumente. To znamená, že pri danej sekvencii trénovacích slov *w1*, *w2*, *w3*, ..., *wT* je cieľom modelu maximalizovať priemernú logaritmickú pravdepodobnosť (Mikolov et al., 2013):

$$(1/T)\ \sum__(t = 1)\hat{}T\ \sum__(-c \leq j \leq c, j \neq 0)\ \log p(w_(t + j) \mid w_t)$$

kde *c* predstavuje veľkosť tréningovej matice. To znamená, že väčšie *c* predstavuje viac tréningových dát, a tým pádom väčšiu presnosť modelu na úkor času trénovania (Mikolov et al., 2013).

1.4.7 GLOVE

Štatistika početnosti slov v znalostnom korpuse je primárny zdroj informácií pre všetky metódy reprezentácie slov. Avšak stále sa naskytuje otázka, ako vyjadriť význam zo spomenutých štatistík a ako výsledné vektory reprezentujú daný význam. Za týmto účelom autori (Pennington et al., 2014) vytvorili model GloVe (Global Vectors).

GloVe je algoritmus učenia bez učiteľa vyvinutý pre generovanie vektorových reprezentácii (embeddingov). Primárnym cieľom je zachytiť sémantické vzťahy medzi slovami (*Glove Word Embedding in NLP - GeeksforGeeks*, 2015).

Tento algoritmus pracuje s maticou výskytu slov, aby sa naučil vektor pre každé slovo a samostatný vektor pre každý kontext (Speer & Chin, 2019).

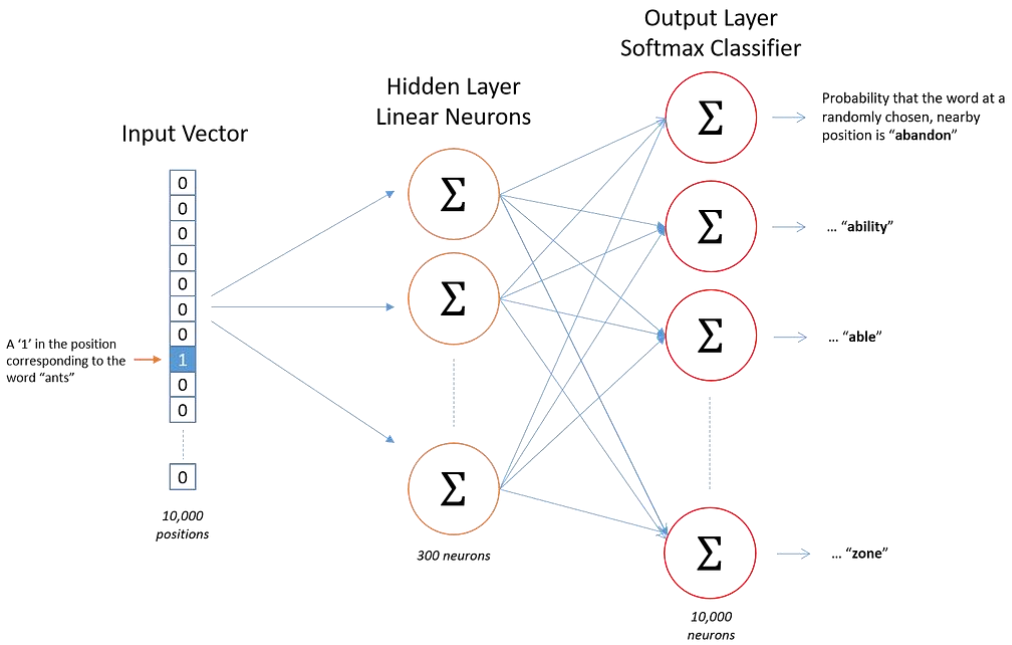
Model *GloVe* je trénovaný na položkách matice slov, ktorá znázorňuje, ako často sa slová vyskytujú spolu v danom korpuse. Cieľom tréningu *GloVe* je naučiť sa vektory slov tak, aby ich skalárny súčin bol rovný logaritmu pravdepodobnosti spoločného výskytu slov (Pennington et al., 2014).

1.4.8 FASTTEXT

Na rozdiel od *Word2Vec* a *GloVe*, ktoré považujú každé slovo za izolované, *FastText* rozdeľuje slová na takzvané n-gramy. To umožňuje generovať vkladanie slov mimo slovnej zásoby. Napríklad slovo *playing* môže byť rozdelené na n-gramy nasledovne: *pla*, *lay*, *ayi*, *yin*, *ing*. Výsledné vloženie slova *playing* je vypočítané ako súčet alebo priemer vložení daných podslov (*FastText Embedding Technique. - Medium*, 2014).

1.4.9 WORD2VEC

Word2Vec vytvára reprezentáciu slov vo forme vektora. Slová používané v podobných kontextoch sú efektívne reprezentované ich blízkosťou vo vektorovom priestore. *Word2Vec* vlastne predstavuje dvojvrstvovú neurónovú sieť. Ako vidíme na obrázku (*Obrázok 4*), skrytá vrstva obsahuje všetky dokumenty v našej trénovacej množine, pričom počet neurónov prítomných v skrytej vrstve je rovnaký ako dĺžka vstupu (Suri, 2022).



Obrázok 4: Príklad neurónovej siete Word2Vec.[4]

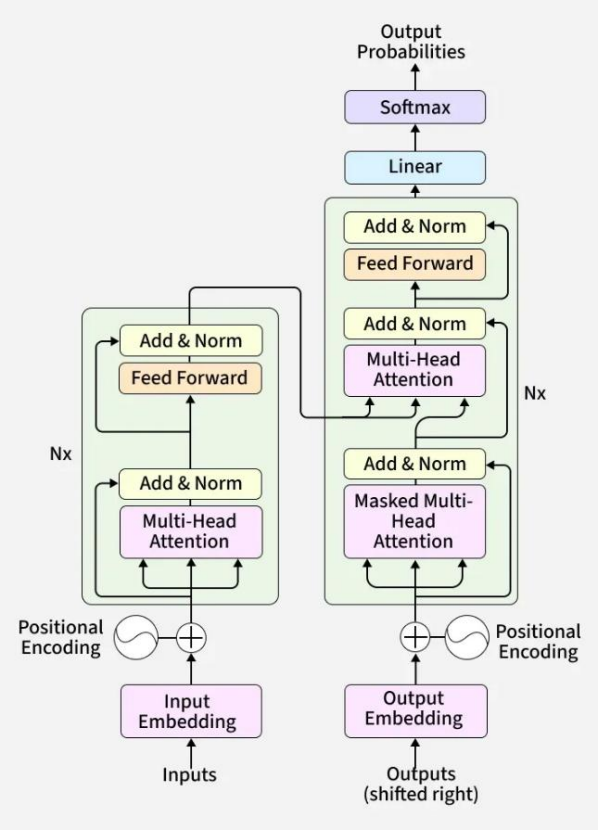
Word2Vec, *GloVe* a *FastText* predstavujú významný pokrok v oblasti textovej reprezentácie, ktoré využívajú okolie slov na generovanie vektorových reprezentácií. Word2Vec sa zameriava na kontext s použitím *Continuous Bag of Words* (*CBOW*) (Cao & SAS France, 2024).

1.5 TRANSFORMERY A MODERNÉ JAZYKOVÉ MODELY

Autori (Vaswani et al., 2023) predstavujú architektúru transformer ako prelomový prístup v NLP, ktorý nahradil rekurentné (*RNN*) a konvolučné modely (*CNN*). Transformer je postavený na mechanizme *self-attention*, ktorý umožňuje

[4] Zdroj Obrázok 4: https://miro.medium.com/v2/resize:fit:1100/format:webp/1*XBaqjqpnBIXtXLzWjkuLQQ.png

modelu spracovávať všetky slová paralelne, čo zásadne zvyšuje rýchlosť a kvalitu spracovania dlhých textov. Architektúru samotného transformera môžeme vidieť na obrázku (*Obrázok 5*).



Obrázok 5: Architektúra transformeru.[5]

Pôvodný transformer model sa skladá z enkódera a dekódera, z ktorých každý je zásobníkom L identických blokov. Každý blok sa skladá zo *self-attention* modulu a pozíciou riadenej *feed-forward* siete. Okolo každého modulu je reziduálne pripojenie, po ktorom nasleduje normalizácia vrstiev (Lin et al., 2022).

1.5.1 BERT A JEHO VARIANTY

BERT (Bidirectional Encoder Representations from Transformers) je založený na transformerovej architektúre, pričom využíva iba jeho enkóder. Ide o hlboko obojsmerné trénovanie, čo znamená, že pri spracovaní textu berie do úvahy kontext z oboch strán súčasne. *BERT* je trénovaný na dvoch úlohách (Devlin et al., 2018):

[5] Zdroj Obrázok 5: https://media.geeksforgeeks.org/wp-content/uploads/20251004124012585570/transformers.webp

- *Masked Language Modeling*,
- *Next Sentence Prediction*.

Masked Language Modeling náhodne maskuje časť slov v texte a predpovedá ich. *Next sentence prediction* učí model rozpoznávať vzťah medzi dvojicami viet. Tento spôsob predtrénovania umožňuje následné doladenie pre veľa NLP úloh, ako klasifikácia viet, označovanie sekvencií, a podobne (Devlin et al., 2018).

1.5.2 ROBERTA

Ide o optimalizovanú a výkonnejšiu verziu modelu *BERT*, ktorá dokazuje, že pôvodné predtrénovanie nebolo optimálne. *RoBERTa* predstavuje nový proces predtrénovania. Ide o odstránenie *Next Sentence Prediction*, použitie väčšieho množstva dát, dlhšie trénovanie a dynamické maskovanie. Vďaka týmto úpravám *RoBERTa* dosahuje lepšie výsledky predtrénovania ako pôvodný model, a to vo všetkých úlohách (Liu et al., 2019).

1.5.3 DISTILBERT

DistilBERT predstavuje rýchlejšiu verziu modelu *BERT*, ktorá zachováva jeho výkon, ale zároveň znižuje nároky na výpočty a pamäť. Tento model vznikol pomocou *knowledge distillation*, ktorý sa učí od pôvodného *BERT* modelu a snaží sa replikovať jeho správanie. Vďaka tomu má *DistilBERT* výrazne menej parametrov a je takmer dvakrát rýchlejší. Je ideálny pre úlohy, kde je dôležitá rýchlosť a efektivita (Sanh et al., 2019).

1.6 ANALÝZA SENTIMENTU

Podľa autorov článku (Sharma & Goyal, 2023) analýza sentimentu predstavuje výpočtový rozbor ľudského správania, perspektív a emócií. Analýza sentimentu zahŕňa podrobnú analýzu textu zameranú na identifikáciu a extrakciu subjektívnych informácií, pričom cieľom NLP analýzy sentimentu je identifikovať, či daný text predstavuje pozitívny, negatívny alebo neutrálny postoj. Napriek tomu, že sa analýza sentimentu môže zdať priamočiara, zahŕňa aj identifikáciu sarkazmu a subjektivity.

Klasifikácia sentimentu je veľmi užitočná vo viacerých smeroch. Napríklad marketing, sociálne siete, recenzie produktov a podobne. Existujú dve hlavné metódy na analýzu sentimentu, a to:

- lexikálna klasifikácia,
- strojové učenie.

Lexikálna klasifikácia vyžaduje preddefinované slovníky sentimentu, pričom klasifikácia s použitím strojového učenia automaticky klasifikuje sentiment na základe tréningových dát (Bonta et al., 2019).

1.6.1 LEXIKÁLNA ANALÝZA SENTIMENTU

Lexikálna analýza využíva takzvané lexikóny sentimentu, ktoré obsahujú informácie o tom, ktoré slová predstavujú pozitívny a ktoré negatívny sentiment. Lexikóny ako *ANEW* (*Affective Norms for English Words*), *SentiWordNet* a *SenticNet* obsahujú slová označené valenciou pre citlivosť sentimentu. Jednou z výhod lexikálnej klasifikácie je, že slovníky sentimentu sa dajú ľahko upraviť alebo doplniť (Bonta et al., 2019).

Lexikálny prístup ku klasifikácii sentimentu zahŕňa vypočítanie orientácie dokumentu na základe sémantickej orientácie slov a fráz (Turney, 2002). Lexikóny sentimentu môžu byť vytvorené manuálne alebo automaticky s použitím takzvaných „seed words“ na rozšírenie slovníka.

Ako uvádza autor (Taboada et al., 2011), väčšina výskumu v lexikálnej analýze sentimentu používa prídavné mená ako indikátory sémantickej orientácie textu. Najskôr sa vytvorí zoznam prídavných mien a im zodpovedajúcich hodnôt sémantickej orientácie. Následne sa pre daný text extrahujú všetky prídavné mená a označia sa hodnotami sémantickej orientácie zo slovníka. Nakoniec sa agregujú výsledky do jedného skóre pre daný text.

1.6.2 SENTIWORDNET

SentiWordNet je výsledkom automatickej anotácie synsetov *Wordnet*-u na pozitívne, negatívne a objektívne. Každý synset *s* je spojený s troma číselnými hodnotami *Pos(s)*, *Neg(s)*, *Obj(s)* (Baccianella et al., 2010).

Ide o trénovanie skupiny klasifikátorov, z ktorých každý dokáže rozhodnúť, či je daný synset pozitívny, negatívny alebo objektívny. Každý klasifikátor sa líši od ostatných trénovacími dátami, čím sa dosahujú rôzne klasifikácie synsetov *Wordnet*-u (Esuli & Sebastiani, 2006).

Verzia *SentiWordNet 3.0* má takmer identický proces generovania. Ide o *semi-supervised* učenie, ktoré sa skladá z hlavných krokov, a to:

- rozšírenie základnej sady synsetov,
- trénovanie klasifikátora.

V prvom kroku sa vezme sada 7 pozitívnych synsetov a 7 negatívnych synsetov, ktoré sa automaticky rozšíria prechádzaním viacerých vzťahov *Wordnet*-u, ktoré môžu zachovať alebo invertovať vlastnosti synsetu.

V druhom kroku sa vygenerované sady kombinujú s ďalšou sadou synsetov, o ktorých sa predpokladá, že majú objektívne hodnotenie. Tieto sady synsetov sú použité ako trénovacie sady pre trénovanie ternárneho klasifikátora.

Nakoniec sú zvyšné synsety *Wordnet*-u postupne pridávané do kroku 2, kde ich klasifikátor označí hodnotami *positive*, *negative*, alebo *objective* (Baccianella et al., 2010).

Testovanie presnosti jednotlivých hodnôt priradených synsetom v slovníku *SentiWordNet* je takmer nemožné, pretože by sme na to potrebovali manuálne ohodnotený slovník *WordNet* na základe troch kritérií, a to: pozitivita, negativita a objektívnosť. Dôvod pre automatické generovanie slovníkov akým je *SentiWordNet* je práve nedostupnosť týchto zdrojov. Približnú spoľahlivosť týchto slovníkov vieme odhadnúť na základe presnosti v skutočných príkladoch analýzy sentimentu (Esuli & Sebastiani, 2006).

1.7 STROJOVÉ UČENIE A ANALÝZA SENTIMENTU

Prístupy strojového učenia sa využívajú na vytvorenie algoritmu a definovanie modelu na základe výberu žiaducich vlastností, ktoré sú naučené z tréningových dát (Bonta et al., 2019). Medzi používané metódy patria *Support Vector Machine* (SVM), *Random forest* a lineárna regresia, pričom ide o algoritmy učenia s učiteľom, ktoré môžeme zaradiť do dvoch kategórií:

- klasifikácia,
- regresia.

Na rozdiel od učenia s učiteľom, algoritmy učenia bez učiteľa nepoužívajú vopred označené tréningové dáta na trénovanie klasifikátora. Medzi najbežnejšie patria *K-Means* a *Apriori*, pričom metóda učenia bez učiteľa sa využíva len zriedka pri analýze sentimentu (Ahmad et al., 2017).

Ako uvádzajú autori (Bonta et al., 2019), vyššie spomenuté algoritmy učenia s učiteľom sa dokážu automaticky naučiť rôzne vlastnosti prostredníctvom optimalizácie. Keďže daný klasifikátor je trénovaný na označených dátach, obvykle sa nehodí na klasifikáciu dát z iných domén. Na prekonanie tejto limitácie sa preto odporúča klasifikácia s použitím lexikónov sentimentu.

1.8 POROVNANIE ONLINE NÁSTROJOV ANALÝZY SENTIMENTU

V nasledujúcej časti analýzy súčasného stavu si popíšeme niekoľko online nástrojov, ktoré sa dajú, mimo iného, použiť aj na analýzu sentimentu a podrobne ich porovnáme prostredníctvom porovnávacej tabuľky.

1.8.1 IBM WATSON NLP

Na analýzu sentimentu v texte použili autori (Carvalho et al., 2019) *IBM Natural Language Understanding* (NLU), čo je skupina služieb dostupná na platforme IBM Cloud. Spomínané NLU je dostupné na stránke IBM, ale je možné ho využívať aj prostredníctvom API. NLU dokáže vypočítať skóre emócií a sentimentu a extrahovať kľúčové slová, kategórie a sémantické príznaky.


```
{
  "sentiment": {
    "score": 0.9255063900060722,
    "mixed": false,
    "label": "positive"
  }
}
```


Pri vstupnej vete *„It is powerful and easy to use and integrate with third party applications.“* dostaneme odpoveď vo formáte JSON, ktorú môžeme vidieť vyššie.

Hodnota *score* sa pohybuje v rozpätí -1 až 1, pričom záporné hodnoty značia negatívny a pozitívne hodnoty značia pozitívny sentiment. Hodnota *label* slovne označuje výsledný sentiment a hodnota *mixed* značí, či sa vo vstupnom texte nachádza kombinácia sentimentov. Táto analýza sentimentu predstavuje deterministický spôsob klasifikácie, čo znamená, že pri danom texte bude výsledok vždy rovnaký (*Detect Sentiment | IBM Cloud Docs*, n.d.).

1.8.2 GOOGLE CLOUD NLP API

Analýza sentimentu Google Cloud NLP skúma daný text a identifikuje prevládajúci emocionálny príznak v texte s cieľom určiť pozitívny, negatívny alebo neutrálny postoj. Analýza sentimentu sa vykonáva prostredníctvom metódy *analyzeSentiment*. Pokiaľ je požiadavka na analýzu úspešná, server vráti odpoveď v JSON formáte (*Cloud Natural Language API | Google Cloud*, 2025):


```
{
  "documentSentiment": {
    "magnitude": 0.8,
    "score": 0.8
  },
  "language": "en",
  "sentences": [
    {
      "text": {
        "content": "Enjoy your vacation!",
        "beginOffset": 0
      },
      "sentiment": {
        "magnitude": 0.8,
        "score": 0.8
      }
    }
  ]
}
```


V odpovedi vidíme, že *documentSentiment.score* vyhodnotilo sentiment s hodnotou *score* 0,8, čo značí pozitívny sentiment. Hodnota *magnitude* reprezentuje emocionálnu intenzitu textu, pričom hodnota nemá horný limit (*Cloud Natural Language API | Google Cloud*, 2025).

1.8.3 AWS COMPREHEND

Amazon Comprehend je NLP služba, ktorá využíva strojové učenie na získavanie rôznych informácií z textu. Poskytuje rozpoznávanie entít, klasifikáciu textu, extrakciu kľúčových slov, analýzu sentimentu prostredníctvom rozhrania API (*Amazon Comprehend – Features*, 2025).


```
{
    "SentimentScore": {
        "Mixed": 0.0033542951568961143,
        "Positive": 0.9869875907897949,
        "Neutral": 0.008563132025301456,
        "Negative": 0.0010949420975521207
    },
```

```
    "Sentiment": "POSITIVE",
  }
}
```

Metóda *DetectSentiment* analyzuje text a vracia výsledok prevažujúceho príznaku sentimentu (pozitívny, negatívny, neutrálny alebo zmiešaný). Keď ako vstup zadáme text "*Today is my birthday, I am so happy.*", server nám vráti odpoveď, ktorá je uvedená vyššie. Hodnota *mixed* predstavuje mieru neistoty pri určovaní polarity sentimentu. V prípade, že hodnoty *positive* a *negative* sú podobné, hodnota *mixed* bude výrazne väčšia, čo značí zmiešaný negatívny a pozitívny sentiment (*DetectSentiment - Amazon Comprehend API*, n.d.).

1.8.4 NLP CLOUD

NLP Cloud poskytuje výkonné modely pre analýzu sentimentu, klasifikáciu, sumarizáciu, extrakciu kľúčových slov a fráz, klasifikáciu zámerov, generovanie textu a obrázkov, strojový preklad, sémantickú podobnosť, tokenizáciu a POS tagging. Je pripravený na produkciu a poskytuje sa prostredníctvom Rest API (*NLP Cloud | LangChain*, 2025). V tabuľke (*Tabuľka 1*) môžeme vidieť priame porovnanie spomínaných nástrojov analýzy sentimentu:

Tabuľka 1: Porovnanie online nástrojov analýzy sentimentu.[6]

Nástroj/Služba	IBM Cloud	Google Cloud	NLP Cloud	AWS Comprehend
Jazyky	13+ jazykov	10+ jazykov	EN, FR, ES, DE a ďalšie	50+ jazykov
Formát dát	Text, HTML, URL	Text, HTML	Text, JSON	Text
Typ analýzy	Strojové učenie, pravidlá, hybrid	Strojové učenie, predtrénované LLM modely	Distilbert Base, Dolphin, LLaMA open-source	Strojové učenie, predtrénované modely
Výhody	Flexibilita	Predtrénované LLM modely	Open-source, prispôsobiteľnosť	Škálovateľnosť
Nevýhody	Vyššia cena pre malé objemy	Môže vyžadovať technické zručnosti	Menej presné v porovnaní s inými službami	Dodatočné náklady pri veľkých objemoch

[6] Zdroje Tabuľka 1: https://cloud.ibm.com/docs/natural-language-understanding, https://cloud.google.com/natural-language/docs, https://docs.aws.amazon.com/comprehend/latest/dg/how-targeted-sentiment.html, https://docs.nlpcloud.com.

2 CIELE ZÁVEREČNEJ PRÁCE

Cieľom tejto diplomovej práce je vytvoriť vektorový slovník sentimentu v slovenskom jazyku a následné trénovanie a vyhodnotenie modelu strojového učenia pre automatickú analýzu sentimentu. Vytvorený model by mal byť schopný analyzovať sentiment (pozitívny, negatívny) pre akýkoľvek vstupný text v slovenskom jazyku.

Čiastkové ciele:

- vybrať vhodný vektorový slovník sentimentu v anglickom jazyku,
- predspracovať získaný slovník sentimentu pre prípravu na preklad,
- vybrať vhodný model strojového učenia na klasifikáciu sentimentu v slovenskom jazyku,
- získať vhodné dáta v slovenskom jazyku na trénovanie vytvoreného modelu,
- zostrojiť slovenský lexikón sentimentu využitím strojového prekladu,
- otestovať preložený slovník sentimentu na vyhodnotenie jeho správnosti,
- predspracovať získané dáta v slovenskom jazyku na trénovanie binárnej klasifikácie sentimentu,
- natrénovať model pre binárnu klasifikáciu sentimentu na pripravenom datasete,
- vyhodnotiť dosiahnuté výsledky.

3 NÁVRH A METODIKA

V tejto kapitole podrobne popíšeme postup pri realizácii definovaných cieľov praktickej časti diplomovej práce. Najskôr definujeme postup pre vytvorenie lexikónu sentimentu pre slovenský jazyk, a neskôr popíšeme vytvorenie a trénovanie modelu strojového učenia pre lexikálnu analýzu sentimentu.

3.1 NÁVRH

V praktickej časti tejto práce budeme priebežne vyhodnocovať čiastkové výsledky. Na tento účel použijeme dataset *Slovak_SA*, ktorý je voľne dostupný na *HuggingFace* (huggingface.com/datasets/DGurgurov/slovak_sa) a obsahuje takmer 3 600 recenzií v slovenskom jazyku označených hodnotou *1* pre pozitívny a *0* pre negatívny sentiment. Vďaka tomu môžeme dataset využiť bez akéhokoľvek upravovania.

Pri realizácii praktickej časti budeme postupovať nasledovne:

- vyberieme vhodný anglický lexikón sentimentu,
- predspracujeme anglický slovník a pripravíme ho na preklad,
- vytvoríme lexikón sentimentu v slovenskom jazyku, pričom zachováme pôvodné hodnoty sentimentu,
- otestujeme vytvorený slovník na vyhodnotenie presnosti/spoľahlivosti na dátach *SlovakSA*,
- vytvoríme model strojového učenia na lexikálnu analýzu sentimentu v texte,
- natrénujeme vytvorený model na datasete recenzii v slovenskom jazyku,
- vyhodnotíme výsledky.

Počas realizácie praktickej časti práce budeme využívať prostredie *Jupyter Notebook* cez rozhranie *Google Colab* na vytvorenie lexikónu sentimentu v slovenskom jazyku, a na následné vytvorenie a trénovanie modelu pre klasifikáciu sentimentu.

3.2 METODIKA VÝVOJA

V metodike vývoja bližšie rozoberieme postup spomenutý v návrhu riešenia. Popíšeme postup od získania potrebných dát, vytvorenie a trénovanie modelu, až po samotné vyhodnotenie dosiahnutých výsledkov.

3.2.1 POROZUMENIE DÁTAM

Pre vytvorenie lexikónu sentimentu v slovenskom jazyku potrebujeme najskôr získať slovník sentimentu pre anglický jazyk. Existuje niekoľko spomínaných slovníkov, ako napríklad:

- SentiWordNet 3.0,
- Sentiment140,
- VADER (Valence Aware Dictionary and Sentiment Reasoner).

My použijeme slovník *SentiWordNet 3.0*, ktorý je voľne dostupný na stiahnutie na *GitHube* (*https://github.com/aesuli/SentiWordNet*). Tento slovník je založený na slovníku *WordNet 3.0* a obsahuje všetky pôvodné synsety označené valenciou sentimentu, a to: *positivity* a *negativity*.

3.2.2 PRÍPRAVA DÁT

Na to, aby sme mohli *SentiWordNet 3.0* preložiť a vytvoriť náš slovenský lexikón sentimentu, musíme ho najskôr transformovať do vhodnej a zrozumiteľnej podoby. *SentiWordNet* obsahuje niekoľko označení, a to:

- POS,
- ID,
- PosScore,
- NegScore,
- SynsetTerms,
- Gloss.

Hodnota *POS* (Part of Speech) definuje gramatickú kategóriu daného synsetu, pričom *n* je podstatné meno, *v* je sloveso, *a* je prídavné meno a *r* je príslovka. *ID* predstavuje číselný identifikátor synsetu *WordNet*-u. *PosScore* a *NegScore* sú hodnoty medzi 0,0 a 1,0, ktoré predstavujú samotné označenie sentimentu. *SynsetTerms* obsahuje jedno alebo viac slov, ktoré majú podobný význam. Hodnota *Gloss* predstavuje krátky popis alebo vysvetlenie významu daného synsetu.

Na obrázku (*Obrázok 6*) môžeme vidieť príklad tabuľky *SentiWordNet*-u so spomínanými atribútmi.

	POS	ID	PosScore	NegScore	SynsetTerms	Gloss
0	a	00001740	0.125	0	able#1	(usually followed by `to') having the necessar...
1	a	00002098	0	0.75	unable#1	(usually followed by `to') not having the nece...
2	a	00002312	0	0	dorsal#2 abaxial#1	facing away from the axis of an organ or organ...
3	a	00002527	0	0	ventral#2 adaxial#1	nearest to or facing toward the axis of an org...
4	a	00002730	0	0	acroscopic#1	facing or on the side toward the apex

Obrázok 6: Príklad tabuľky SentiWordNet.[7]

Súčasťou prípravy dát je taktiež príprava datasetu na trénovanie modelu strojového učenia. Na tento účel sme vybrali dataset *SlovakSA*, ktorý obsahuje takmer 3600 recenzií v slovenskom jazyku ohodnotené binárnou klasifikáciou sentimentu. Dataset obsahuje samotný text recenzie a hodnotu *label*, ktorá obsahuje ohodnotenie sentimentu, kde 1 reprezentuje pozitívnu a 0 negatívnu polaritu. Na obrázku (*Obrázok 7*) môžeme vidieť ukážku spomínaného datasetu.

	label	text
0	1	Bola ústretová,milá a komunikatívna .Šikovná s...
1	1	dobrá obsluha, pekná baba.
2	1	Prijemne vystupovanie, ochota pomôcť
3	1	Za ochotu
4	1	Ďakujem za rýchle vybavenie a ústretovosť

Obrázok 7: Príklad tabuľky SlovakSA.[8]

3.2.3 MODELOVANIE

Pred vytvorením a trénovaním samotného modelu strojového učenia na binárnu klasifikáciu sentimentu musíme najskôr získať lexikón sentimentu pre slovenský jazyk. Využijeme litovský *SlovakWordNet*, ktorý je voľne dostupný (https://korpus.sk/korpusy-a-databazy/databazy/wordnet/). Vďaka tomu, že *SlovakWordNet* a *SentiWordNet* sú založené na slovníku *WordNet*, môžeme existujúce synsety *SlovakWordNet*-u použiť pri vytváraní lexikónu sentimentu. Keďže *SlovakWordNet* neobsahuje všetky synsety *SentiWordNet*u, skombinujeme ho so strojovým prekladom. Na tento účel použijeme *Helsinki-NLP*, ktorý využíva

[7] Zdroj Obrázok 6: Autor.
[8] Zdroj Obrázok 7: Autor.

transformerový model *MarianNMT* na strojový preklad rôznych jazykov vrátane slovenčiny. Po preklade všetkých synsetov slovníku *SentiWordNet* vyhodnotíme správnosť a spoľahlivosť výsledného slovníka sentimentu.

Na klasifikáciu sentimentu využijeme okrem iného transformerový model *RoBERTa*, ktorý budeme trénovať na datasete *SlovakSA*. Model bude upravený na binárnu klasifikáciu sentimentu (pozitívny/negatívny) v slovenskom jazyku, pričom urobíme porovnanie s upraveným a základným modelom *RoBERTa*. Vstupom nášho hybridného modelu budú tokenizované a lematizované texty zo *SlovakSA*, pričom budú obohatené o hodnoty sentimentu získané z vytvoreného lexikónu (*lexical features*).

Tréning modelu prebehne v prostredí *Google Colab*, kde využijeme dostupnú grafickú akceleráciu pre rýchlejšie spracovanie dát.

3.2.4 TESTOVANIE

Cieľom vyhodnotenia výsledkov je overiť kvalitu a spoľahlivosť natrénovaných modelov, ktoré budeme trénovať na binárnu klasifikáciu sentimentu s použitím datasetu *SlovakSA*. Na vyhodnotenie modelov a priebežných výsledkov použijeme metriky binárnej klasifikácie sentimentu, a to:

- Accuracy,
- Precision,
- Recall,
- F1-Score.

Použitím daných metrík dokážeme posúdiť spoľahlivosť a správnosť trénovaného modelu a vytvoreného slovníka sentimentu, pričom *accuracy* predstavuje mieru správnych klasifikácií, *precision* presnosť klasifikácie, *recall* schopnosť identifikovať všetky pozitívne prípady a *F1-Score* predstavuje priemer hodnôt *precision* a *recall*.

Keďže pri modelovaní používame strojovo preložený slovník sentimentu, je dôležité vyhodnotiť správnosť a spoľahlivosť vytvoreného slovníka. Tento spôsob prekladu môže priniesť nedostatky spojené s nesprávnym priradením sentimentu pri viacvýznamových slovách alebo vytvorením prekladových chýb, ktoré môžu následne negatívne ovplyvniť kvalitu trénovaného modelu a výsledného slovníka sentimentu.

4 VÝSLEDKY

V tejto časti záverečnej práce podrobne rozoberieme jednotlivé kroky implementácie cieľov a čiastkových cieľov práce. Počas realizácie praktickej časti budeme priebežne testovať čiastočné výsledky. Nakoniec vyhodnotíme dosiahnuté výsledky, pričom porovnáme jednotlivé prístupy klasifikácie sentimentu.

4.1 PRÍPRAVA DÁT

Najskôr sme vytvorili slovenský lexikón sentimentu s použitím *SentiWordNet*-u, *SlovakWordNet*-u a strojového prekladu. Namapovali sme existujúce synsety zo slovenského *WordNet*-u priamo na *SentiWordNet* a zvyšné synsety strojovo preložili. Tým sme získali slovenský *SentiWordNet*, ktorý obsahuje všetky pôvodné synsety SWN v slovenskom jazyku.

V prvom kroku sme importovali a predspracovali slovník *SentiWordNet 3.0*, ktorý je voľne dostupný na platforme *GitHub* (*https://github.com/aesuli/SentiWordNet*). Pozrime sa na časť kódu, uvedenú nižšie:

```
import pandas as pd

file_path = "SentiWordNet_3.0.0.txt"
rows = []

with open(file_path, "r", encoding="utf-8") as f:
      for line in f:
            if line.startswith("#"): #ignore lines starting with '#'.
                  continue
            #strip the input line of spaces and split it on indent.
            parts = line.strip().split("\t")
            if len(parts) != 6:
                  continue
            #define parts of each line
            pos, sid, pos_score, neg_score, synset_terms, gloss = parts
            #append each line to a list as a dictionary entry
            rows.append({
                  "pos": pos,
                  "sid": str(sid).zfill(8),
                  "pos_score": float(pos_score),
                  "neg_score": float(neg_score),
                  "obj_score": 1.0 - (float(pos_score) + float(neg_score)),
                  "synset_terms": synset_terms,
                  "gloss": gloss
            })
```

Keďže *SentiWordNet* je dostupný vo forme textového súboru, bolo potrebné ho najskôr transformovať do *Pandas DataFrame* pre jednoduchú manipuláciu. Postupne po riadkoch sme načítali textový súbor SWN, pričom sme preskočili všetky popisné riadky, ktoré začínajú mriežkou (#). Časti, ktoré sú oddelené tabulátorom a sú v počte 6, sme riadok po riadku vložili do zoznamu *rows*. K nim sme priradili *objective score*, ktoré sme získali výpočtom na základe *pos_score* a *neg_score*.

```
#create a dataframe based on 'rows' list
swn_syn = pd.DataFrame(rows)
#create a unique id for each synset
swn_syn["synset_id"] = swn_syn["pos"] + ":" + swn_syn["sid"]
swn_syn = swn_syn.drop_duplicates("synset_id").set_index("synset_id")
swn_syn.head()
```

Nakoniec sme použili hotový zoznam *rows* na vytvorenie *DataFrame*-u, a vytvorili sme unikátne *synset_id* pre každý synset. Na obrázku (*Obrázok 8*) môžeme vidieť výslednú tabuľku.

	pos	sid	pos_score	neg_score	obj_score	synset_terms	gloss
synset_id							
a:00001740	a	00001740	0.125	0.00	0.875	able#1	(usually followed by `to') having the necessar...
a:00002098	a	00002098	0.000	0.75	0.250	unable#1	(usually followed by `to') not having the nece...
a:00002312	a	00002312	0.000	0.00	1.000	dorsal#2 abaxial#1	facing away from the axis of an organ or organ...
a:00002527	a	00002527	0.000	0.00	1.000	ventral#2 adaxial#1	nearest to or facing toward the axis of an org...
a:00002730	a	00002730	0.000	0.00	1.000	acroscopic#1	facing or on the side toward the apex

Obrázok 8: Výsledná tabuľka SentiWordNet.[9]

Ďalej sme načítali litovský *SlovakWordNet*, predspracovali ho a rovnako ako *SentiWordNet*, a previedli ho na *DataFrame*. Pozrime sa na časť kódu:

```
import re

sk_path = "sk-wn-2013-01-23.txt"

rows = []
#find RS block regex
pattern = re.compile(r"ʀs\s*(\d+)\s+\d+\s+([nvarsa])\b")

with open(sk_path, "r", encoding="utf-8") as f:
    for line in f:
```

[9] Zdroj Obrázok 8: Autor.

```
        parts = line.rstrip("\n").split("\t")

        if len(parts) < 4:
            continue

        #define parts of each line
        lex_num, pos_local, lemmas_sk, RS_raw = parts[:4]
        offset = None
        pos_raw = None

        #find RS marker in each line and assign offset and pos_raw
        for cell in parts:
            m = pattern.search(cell)
            if m:
                offset, pos_raw = m.groups()
                break

        if offset is None:
            continue

        offset = offset.zfill(8)
        pos = "a" if pos_raw == "s" else pos_raw

        rows.append({
            "lex_num": int(lex_num),
            "pos_local": pos_local,
            "lemmas_sk": lemmas_sk,
            "RS_block": RS_raw,
            "pwn_offset": offset,
            "pwn_pos_raw": pos_raw,
            "pwn_pos": pos,
            "synset_id": f"{pos}:{offset}"
        })

sk = pd.DataFrame(rows)
sk.head()
```

Slovenský *WordNet* má iné formátovanie ako *SentiWordNet*, preto sme ho museli upraviť tak, aby sa dal neskôr namapovať na pôvodný *SentiWordNet*. Najskôr sme rozdelili jednotlivé riadky na časti (*parts*) na základe tabulátora a odstránili sme znaky nových riadkov (*\n*). Potom sme vyhľadali znak ᴿˢ (*record separator*) v každom riadku, čím sme rozdelili jednotlivé časti *WordNet*-u. Následne sme s použitím regulárneho výrazu extrahovali *offset* a *part of speech*, na základe čoho sme vytvorili *synset_id*, ktoré sa zhoduje s formátom *SentiWordNet*. Nakoniec sme vytvorili samotný *DataFrame*. Na obrázku (*Obrázok 9*) vidíme výslednú tabuľku.

	lex_num	pos_local	lemmas_sk	RS_block	pwn_offset	pwn_pos_raw	pwn_pos	synset_id
0	1999	a	abnormálny; nadmerný; priveľký	RS 01533535 00 s 01 abnormal 0 001 & 01533120 ...	01533535	s	a	a:01533535
1	31	a	absolútny; oprávnený; zákonný; zákonom ustanovený	RS 00557108 00 s 01 vested 0 001 & 00556709 a ...	00557108	s	a	a:00557108
2	28	a	absolútny; zvrchovaný	RS 00719328 00 s 01 absolute 0 002 & 00718924 ...	00719328	s	a	a:00719328
3	2204	a	abstrakčný; schopný abstrahovať; schopný abstr...	RS 00860932 00 s 01 abstractive 0 003 & 008606...	00860932	s	a	a:00860932
4	35	a	absurdný; nezmyselný	RS 01431112 00 s 01 absurd 0 003 & 01430847 a ...	01431112	s	a	a:01431112

Obrázok 9: Výsledná tabuľka SlovakWordNet.[10]

4.1.2 Mapovanie synsetov SlovakWordNet na SentiWordNet 3.0

Máme predspracovaný *SentiWordNet* a *SlovakWordNet* a ďalším krokom je samotné mapovanie existujúcich synsetov zo slovenského *WordNet*-u na *SentiWordNet*. Na tento účel sme najskôr zmenili formátovanie slovenských lem, aby zodpovedali formátu *SentiWordNet*.

Najskôr sme rozdellili jednotlivé slovenské lemy podľa bodkočiarky, a nahradili medzery znakom (_), ako vidíme na nasledujúcej časti kódu:

```
#split lemmas function
def split_sk_lemmas(x):
      if not isinstance(x, str):
            return []
            out = []
            for t in x.split(";"):
            t = t.strip()
            if t:
                  out.append(t.replace(" ", "_"))
      return out

sk["lemmas_sk_list"] = sk["lemmas_sk"].apply(split_sk_lemmas)
```

Teraz náš dataset *sk* obsahuje atribút *lemmas_sk_list*, ktorý obsahuje zoznamy jednotlivých lem extrahovaných zo stĺpca *lemmas_sk*. Ďalší krok pozostáva z transformácie vytvorených zoznamov na jeden *string* vo formáte *SentiWordNet*. Pozrime sa na nasledujúcu časť kódu:

```
#aggregate lemmas
sk_agg = sk.groupby("synset_id")["lemmas_sk_list"].sum().to_frame()
cleaned = []

for lemma in sk_agg["lemmas_sk_list"]:
    cleaned.append(sorted(set(lemma)))
```

[10] Zdroj Obrázok 9: Autor.

```
sk_agg["lemmas_sk_list"] = cleaned
```

Najskôr sme zoskupili všetky lemy, ktoré majú identické *synset_id* a zoradili sme ich abecedne. Následne sme ku každej leme pridali takzvané „*sense numbers*“, ktoré reprezentujú poradie významov jednotlivých synsetov. Tieto *sense* čísla pochádzajú priamo z *WordNet*-u a pre naše účely klasifikácie sentimentu sú nepotrebné. Preto sme pre každú lemu aplikovali *sense number #1*, aby sme zachovali správne formátovanie.

```
#add "#1" sense number
terms = []
for lemmas in sk_agg["lemmas_sk_list"]:
    if not lemmas:
        terms.append(None)
    else:
        s = ""
        for lemma in lemmas:
            s += lemma + "#1 "
        terms.append(s.strip())

sk_agg["synset_terms_sk"] = terms
```

Nakoniec sme namapovali samotné synsety na *SentiWordNet*. Ako vidíme na časti kódu, ktorá je uvedená nižšie, mapovali sme nasledovne:

```
swn_out = swn_syn.join(sk_agg[["synset_terms_sk"]], how="left")
swn_out["synset_terms_en"] = swn_out["synset_terms"]
#replace where Slovak exists
swn_out["synset_terms"]=
swn_out["synset_terms_sk"].fillna(swn_out["synset_terms"])
```

Výsledkom je dataset *SentiWordNet*, ktorý obsahuje kombináciu slovenských a anglických synsetov. Celkovo sme nahradili 17 966 synsetov z celkových 117 629. Zvyšné synsety sme preložili strojovo s použitím *HelsinkiNLP*, ako si ukážeme v ďalšej podkapitole.

4.1.3 Preklad synsetov SentiWordNet 3.0

Zvyšné synsety *SentiWordNet*-u v anglickom jazyku sme preložili do slovenského jazyka, čím sme dosiahli kompletné obsiahnutie synsetov *SentiWordNet*-u. Aby sme zbytočne neprekladali synsety, ktoré sme v predchádzajúcom kroku nahradili,

museli sme najskôr odlíšiť slovenské od anglických synsetov. Preto sme rozdelili náš upravený *SentiWordNet DataFrame* na dve časti. Postupovali sme nasledovne:

```
swn_mapped = (
      swn_out[swn_out["synset_terms_sk"].notna()]
      .drop(columns=["synset_terms_sk", "synset_terms_en"])
      .copy()
)

print("Mapped synsets:", len(swn_mapped))
swn_mapped.head()
```

Najskôr sme vybrali len tie riadky datasetu, ktoré obsahujú slovenské lemy (*synset_terms_sk*) a zároveň sme odstránili pomocné atribúty. Výsledkom je dataset, ktorý obsahuje len namapované lemy zo slovenského *WordNet*-u. V ďalšom kroku sme spravili to isté pre nenamapované lemy, ako môžeme vidieť nižšie:

```
swn_unmapped = (
      swn_out[swn_out["synset_terms_sk"].isna()]
      .drop(columns=["synset_terms_sk", "synset_terms_en"])
      .copy()
)

print("Unmapped synsets:", len(swn_unmapped))
swn_unmapped.head()
```

Ďalej sme aplikovali *HelsinkiNLP* strojový preklad na dataset *swn_unmapped* a nakoniec sme ich spojili dokopy do jedného celku. Najskôr sme načítli potrebný model a následne definovali funkcie na samotný preklad. Pozrime sa na časť kódu:

```
from transformers import MarianMTModel, MarianTokenizer
import torch

model_name = "Helsinki-NLP/opus-mt-en-sk"

tokenizer = MarianTokenizer.from_pretrained(model_name)
model = MarianMTModel.from_pretrained(model_name)

device = "cuda" if torch.cuda.is_available() else "cpu"
model.to(device)
```

Inicializovali sme potrebný model, pričom sme využili GPU akceleráciu, čo nám výrazne zrýchlilo samotný preklad synsetov. Ďalej sme pokračovali definíciou funkcií na samotný preklad:

```
#translate a batch of lemmas
def mt_batch(texts, tokenizer, model, device, batch_size=64, max_length=64,
log_every=100):
    outs = []
    total_batches = (len(texts) + batch_size - 1) // batch_size

    for i in range(0, len(texts), batch_size):
        batch_id = i // batch_size + 1
        batch = texts[i:i+batch_size]

        enc = tokenizer(
            batch,
            return_tensors="pt",
            padding=True,
            truncation=True,
            max_length=max_length
        ).to(device)

        with torch.no_grad():
            gen = model.generate(**enc, max_length=max_length)

        outs.extend(tokenizer.batch_decode(gen, skip_special_tokens=True))

        #print progress regularly
        if batch_id % log_every == 0 or batch_id == total_batches:
            print(f"[MT] Batch {batch_id}/{total_batches} completed")

    return outs
```

Vyššie uvedená funkcia má za úlohu preklad jednotlivých dávok lem (batches). Najskôr rozdelí vstupné lemy na dávky vo veľkosti 64 lem, zakóduje ich pre prekladový model, preloží a nakoniec vráti preložené lemy v rovnakom poradí, v akom prišli na vstupe.

Najskôr sme odstránili formátovanie *SentiWordNet*-u, aby sme na vstupe mali čisto zoznam lem. Na tento účel sme vytvorili funkciu, ktorú vidíme nižšie:

```
#transform, translate, repack
def translate_synset_terms_series(terms_series, tokenizer, model, device,
batch_size=64):
    #fill empty and split synset terms
```

```
    terms_series = terms_series.fillna("")
    token_lists = terms_series.apply(lambda s: s.split() if s else [])

    lemmas = []
    senses = []
    #remove and save sense numbers
    for toks in token_lists:
        for t in toks:
            if "#" in t:
                w, sn = t.rsplit("#", 1)
            else:
                w, sn = t, "1"
            lemmas.append(w.replace("_", " "))
            senses.append(sn)

    if not lemmas:
        return terms_series.copy()
    #translate batch
         lemmas_sk    =    mt_batch(lemmas,    tokenizer,    model,    device,
batch_size=batch_size, max_length=64)
    lemmas_sk = [x.strip().replace(" ", "_") for x in lemmas_sk]

    out_rows = []
    k = 0
    #readd sense numbers and join into one string
    for toks in token_lists:
        new_toks = []
        for _ in toks:
            new_toks.append(f"{lemmas_sk[k]}#{senses[k]}")
            k += 1
        out_rows.append(" ".join(new_toks))

    return pd.Series(out_rows, index=terms_series.index)
```

Funkcia *translate_synset_terms_series* najskôr rozdelí synsety podľa medzier, odstráni a uloží *sense numbers* do premennej *senses* a následne zavolá funkciu *mt_batch*, ktorá vykoná samotný preklad. Po preložení všetkých synsetov nasleduje opätovné pridanie *sense numbers* a zlúčenie do jedného reťazca.

```
# translate the unmapped set
swn_unmapped = swn_unmapped.copy()
swn_unmapped["synset_terms_sk_mt"] = translate_synset_terms_series(
    swn_unmapped["synset_terms"],
    tokenizer=tokenizer,
    model=model,
    device=device,
    batch_size=64
)
```

```
#replace original synset_terms
swn_unmapped["synset_terms"] = swn_unmapped["synset_terms_sk_mt"]
swn_unmapped = swn_unmapped.drop(columns=["synset_terms_sk_mt"])
swn_unmapped["synset_terms"].head()
```

Ako môžeme vidieť vyššie, inicializovali sme samotný preklad a následne sme nahradili pôvodné synsety preloženými. Nakoniec sme zlúčili namapovaný a preložený dataset a exportovali sme ho ako textový súbor vo formáte *SentiWordNet*.

```
swn_final = pd.concat([swn_mapped, swn_unmapped], axis=0)

print("Final rows:", len(swn_final))

swn_final = swn_final.sort_index()
swn_final.head()
```

Cez funkciu *concat* sme zlúčili *swn_mapped* a *swn_unmapped*, a resetovali sme index. Nakoniec sme spravili konečné úpravy a exportovali sme finálny lexikón ako textový súbor, čo môžeme vidieť nižšie:

```
export_cols = ["pos", "sid", "pos_score", "neg_score", "synset_terms",
"gloss"]

#safety checks
missing = [c for c in export_cols if c not in swn_final.columns]
if missing:
    raise ValueError(f"Missing columns in swn_final: {missing}")

out = swn_final.reset_index(drop=True).copy()

#formatting
out["pos"] = out["pos"].astype(str)
out["sid"] = out["sid"].astype(str).str.zfill(8)
out["pos_score"] = out["pos_score"].astype(float)
out["neg_score"] = out["neg_score"].astype(float)

#restore header
header_lines = [
    "# SentiWordNet 3.0.0 – Slovak version",
    "# SynsetTerms localized to Slovak (Slovak WordNet + MT fallback)",
    "# Gloss kept in original English",
    "#",
    "# Format:",
    "# POS\tID\tPosScore\tNegScore\tSynsetTerms\tGloss"
]
```

```
#export to txt
out_path = "SentiWordNet_3.0.0_SK.txt"

with open(out_path, "w", encoding="utf-8") as f:
    for line in header_lines:
        f.write(line + "\n")
    out[export_cols].to_csv(f, sep="\t", index=False, header=False)

out_path
```

Výsledkom celého procesu je kompletná slovenská verzia slovníku *SentiWordNet 3.0* so zachovanými hodnotami sentimentu. Výsledný slovník, ako aj celý zdrojový kód, sú dostupné na *Githube* (*Príloha A*).

4.1.4 Testovanie lexikónu bez lematizácie

V tomto kroku sme otestovali, či je vytvorený slovník vhodný na lexikálnu klasifikáciu v slovenskom jazyku. Toto testovanie prebehlo na datasete *SlovakSA*, ktorý obsahuje recenzie v slovenskom jazyku ohodnotené sentimentom. Testovanie prebiehalo v dvoch krokoch, kde sme otestovali *SlovakSA* bez lematizácie a s lematizáciou.

Očakávame, že lematizovaný dataset dosiahne podstatne lepšie výsledky, keďže lexikón sentimentu obsahuje synsety v základných tvaroch, čo znamená viacej zásahov lexikónu.

```
splits = {'train': 'train.csv', 'validation': 'dev.csv', 'test': 'test.csv'}
sa = pd.read_csv("hf://datasets/DGurgurov/slovak_sa/" + splits["train"])
sa.head()

splits = {
      "train": "train.csv",
      "validation": "dev.csv"
}

sa = pd.concat(
      [
            pd.read_csv(f"hf://datasets/DGurgurov/slovak_sa/{fname}")
            for fname in splits.values()
      ], ignore_index=True
)
print(sa.shape)
sa.head()
```

Načítali sme *SlovakSA* z *HuggingFace*, pričom sme spojili *train* a *val* datasety, ktoré použijeme na testovanie. Ďalej sme pripravili slovník *SlovakSentiWordNet* na preklad:

```
file_path = "SentiWordNet_3.0.0_SK.txt"

rows = []
with open(file_path, "r", encoding="utf-8") as f:
      for line in f:
            if not line.strip() or line.startswith("#"): #skip lines
      containing comments
            continue

            parts = line.rstrip("\n").split("\t", maxsplit=5) #split lines 5
      times

            if len(parts) == 5: #if gloss is missing
                  pos, sid, pos_score, neg_score, synset_terms = parts
                  gloss = ""
            elif len(parts) == 6: #gloss is present
                  pos, sid, pos_score, neg_score, synset_terms, gloss = parts
            else:
                  #safety check
                  continue

            pol = float(pos_score) - float(neg_score) #calculate word
      polarity

            for tok in synset_terms.split():
                  word = tok.rsplit("#", 1)[0].lower() #split sense numbers
                  rows.append((word, pol))
```

V prvom rade sme po riadkoch načítali slovenský *SentiWordNet*, pričom sme odstránili všetky komentáre, rozdelili riadky na 6 častí a uložili výsledok do zoznamu *rows* vo formáte: slovo, polarita.

```
lex_df = pd.DataFrame(rows, columns=["word", "polarity"]) #create a dataframe
using extracted rows
word_lex = lex_df.groupby("word", as_index=False)["polarity"].mean() #group
matching rows
lex_dict = dict(zip(word_lex["word"], word_lex["polarity"])) #convert into
dictionary
```

Nakoniec sme uložili zoznam *rows* ako *DataFrame*, ktorý sme následne konvertovali na *dictionary*, ktorý sme priamo použili na nachádzanie zhôd v dátach.

```
#regex to define allowed symbols. We exclude emojis, punctuation,..
token_re = re.compile(r"[0-9A-Za-zÁÄČĎÉÍĹĽŇÓÔŘŠŤÚÝŽáäčďéíĺľňóôřšťúýž_]+")

def score_text(text):
    if not isinstance(text, str):
        return 0.0, 0

    #parse every token with regex
    toks = token_re.findall(text.lower())
    #find a match in lexicon and save values
    vals = [lex_dict[t] for t in toks if t in lex_dict]
    return (float(np.sum(vals)) if vals else 0.0), len(vals)
```

Ďalej sme zadefinovali funkciu, ktorá postupne prejde celý text *SlovakSA* cez regulárny výraz a následne nájde všetky zhody v slovníku, pričom spočíta všetky hodnoty sentimentu. Tým získame celkovú hodnotu sentimentu pre daný text.

```
#apply score_text
sa = sa.copy()
sa[["lex_score", "lex_hits"]] = sa["text"].apply(
    lambda t: pd.Series(score_text(t))
)

#evaluate results
print(sa[["lex_score", "lex_hits"]].describe())
print("Coverage (>=1 hit):", (sa["lex_hits"] > 0).mean())
```

Funkciu *score_text* sme aplikovali a vypísali sme výsledky. Na výsledkoch vidíme, že priemerný počet zásahov lexikónu je 5 slov, pričom približne 90% textov obsahuje aspoň jeden zásah. Vzhľadom k tomu, že dáta *SlovakSA* neboli lematizované je to pozitívny výsledok. Nakoniec sme ešte vypočítali metriky *accuracy*, *precision*, *recall* a *F1 Score*. Výsledok môžeme vidieť na obrázku (*Obrázok 10*).

```
Accuracy: 0.5938202247191011
Precision: 0.9220085470085471
Recall: 0.5703899537343027
F1: 0.7047774601878317
Confusion matrix [[TN FP], [FN TP]]:
 [[ 388  146]
 [1300 1726]]
```

Obrázok 10: Výsledné metriky testovania bez lematizácie.[11]

[11] Zdroj Obrázok 10: Autor.

Na výsledných metrikách vidíme, že *accuracy* je približne 59%, čo je pochopiteľné, keďže neberieme do úvahy kontext a iróniu. *Precision* je 92%, čo značí vysokú presnosť klasifikácie pozitívnych textov. Hodnota *recall* hovorí, že lexikón zachytáva 57% pozitívnych textov. V ďalšom kroku spravíme lematizáciu dát *SlovakSA* a otestujeme lexikón ešte raz. Očakávame zlepšenie vo všetkých metrikách oproti dátam bez lematizácie.

4.1.5 Testovanie lexikónu s lematizáciou

Na lematizáciu dát SlovakSA sme použili tokenizér stanza. Najskôr sme ho inicializovali a následne sme lematizovali všetok text *SlovakSA* po jednotlivých vetách.

```
def score_text_lemma(text: str):
    if not isinstance(text, str) or not text.strip():
        return 0.0, 0

    #lemmatize tokens
    doc = nlp(text)
    lemmas = []
    for sent in doc.sentences:
        for w in sent.words:
            if w.lemma:
                lemmas.append(w.lemma.lower().replace(" ", "_"))

    #save and sum values
    vals = [lex_dict[l] for l in lemmas if l in lex_dict]
    score = float(np.sum(vals)) if vals else 0.0
    return score, len(vals)
```

Funkcia *score_text_lemma* najskôr rozdelí text na jednotlivé vety a následne lematizuje jednotlivé slová. Nakoniec nájde všetky zhody v lexikóne a vráti výsledok vo formáte: skóre, počet zásahov.

```
#apply score_text_lemma to temporary df
tmp = sa["text"].apply(lambda t: pd.Series(score_text_lemma(t)))
sa = sa.copy()
sa["lex_score_lemma"] = tmp[0]
sa["lex_hits_lemma"] = tmp[1]
```

Funkciu sme aplikovali na celý dataset *SlovakSA*, pričom sme výsledok uložili do *DataFrame tmp*. Ďalej sme vytvorili nové stĺpce v datasete *SlovakSA*, ktorým sme priradili hodnoty skóre a počet zásahov.

```
#binary clasification
y_true = sa["label"].astype(int).values
y_pred = (sa["lex_score_lemma"] > 0).astype(int).values

#calculate metrics
acc = accuracy_score(y_true, y_pred)
prec, rec, f1, _ = precision_recall_fscore_support(y_true, y_pred,
average="binary", zero_division=0)
cm = confusion_matrix(y_true, y_pred)

print("Coverage (>=1 hit):", coverage)
print("Accuracy:", acc)
print("Precision:", prec)
print("Recall:", rec)
print("F1:", f1)
print("Confusion matrix [[TN FP],[FN TP]]:\n", cm)
```

Nakoniec sme vypočítali výsledné metriky a vypísali sme ich. Na obrázku (*Obrázok 11*) môžeme vidieť výsledok.

```
Accuracy: 0.7002808988764045
Precision: 0.9166312207571247
Recall: 0.7121612690019828
F1: 0.801562209410452
Confusion matrix [[TN FP],[FN TP]]:
 [[ 338  196]
 [ 871 2155]]
```

Obrázok 11: Výsledné metriky testovania po lematizácii.[12]

Z výsledkov vidíme, že takmer všetky metriky ukazujú zlepšenie oproti dátam bez lematizácie. Hodnota *accuracy* sa výrazne zlepšila, približne o 11%. *Precision* je porovnateľná s hodnotou bez lematizácie. Pri hodnote *recall* taktiež pozorujeme výrazné zlepšenie o 14%.

Lematizované dáta predstavujú výrazné zlepšenie oproti dátam bez lematizácie. Výsledok výrazne preferuje pozitívnu triedu z dôvodu zaujatosti vstupných dát *SlovakSA*. Na základe testovania môžeme s istotou povedať, že hodnoty sentimentu boli zachované aj po strojovom preklade. Vytvorený lexikón teda predstavuje dobrý základný *baseline* pre lexikálnu klasifikáciu sentimentu.

[12] Zdroj Obrázok 11: Autor.

4.2 MODELOVANIE

V tejto kapitole popíšeme modelovanie záverečnej práce. Najskôr sa zameriame na model *RoBERTa* a neskôr jeho vstup rozšírime o lexikálne *features*, čím vytvoríme hybridný model *RoBERTa*. Nakoniec tieto dva modely porovnáme.

4.2.1 Základný model RoBERTa

V prvom rade sme načítali dáta *SlovakSA* z *HuggingFace*, pričom sme jednotlivé datasety uložili ako *train_df*, *val_df*, *test_df*. Následne sme tieto datasety lematizovali s použitím tokenizéra *Stanza*. Ďalej prejdeme na prípravu a samotné trénovanie modelu *RoBERTa*.

```
from datasets import Dataset, DatasetDict

dset = DatasetDict({
    "train": Dataset.from_pandas(train_df, preserve_index=False),
    "validation": Dataset.from_pandas(val_df, preserve_index=False),
    "test": Dataset.from_pandas(test_df, preserve_index=False),
})

dset
```

V prvom rade sme konvertovali dáta *SlovakSA* z *DataFrame* na *DatasetDict*, ktorý sme použili ako vstup pre tokenizér *RoBERTa*. Na obrázku (*Obrázok 12*) môžeme vidieť výsledný *DatasetDict*.

```
DatasetDict({
    train: Dataset({
        features: ['label', 'text'],
        num_rows: 3560
    })
    validation: Dataset({
        features: ['label', 'text'],
        num_rows: 522
    })
    test: Dataset({
        features: ['label', 'text'],
        num_rows: 1042
    })
})
```

Obrázok 12: Výsledný DatasetDict SlovakSA.[13]

[13] Zdroj Obrázok 12: Autor.

Ďalej sme dáta tokenizovali s tokenizérom *RoBERTa*, ktorý z textu vytvoril *attention_mask* a *input_ids*, ktoré model používa ako vstupné dáta.

```
MODEL_NAME = "TUKE-KEMT/slovak-roberta-base"
#use RoBERTa tokenizer to ensure compatibility
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

def tokenize(batch):
      return tokenizer(
            batch["text"],
            truncation=True,
            max_length=256
      )

      tok = dset.map(tokenize, batched=True)
```

Keďže vytvorený *attention_mask* a *input_ids* nemusia mať vždy rovnakú dĺžku, použili sme *collator*, ktorý doplnil chýbajúce hodnoty nulami, čím sa nám podarilo zabezpečiť jednotný vstup pre model. Časť kódu môžeme vidieť nižšie:

```
MODEL_NAME = "TUKE-KEMT/slovak-roberta-base"

#import model
model = AutoModelForSequenceClassification.from_pretrained(
      MODEL_NAME,
      num_labels=2 #binary classification
)
      #prepare collator for input padding
      collator = DataCollatorWithPadding(tokenizer)
```

Ďalej sme pripravili funkciu *compute_metrics*, ktorú použijeme na počítanie metrík ako počas tréningu modelu, tak aj na vyhodnotenie.

```
acc = evaluate.load("accuracy")
f1  = evaluate.load("f1")

#function to calculate metrics for model evaluation
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    return {
            "accuracy": acc.compute(predictions=preds,
            references=labels)["accuracy"],
            "f1": f1.compute(predictions=preds, references=labels,
            average="binary")["f1"],
    }
```

Pred spustením samotného trénovania sme zadefinovali parametre tréningu, ako aj samotný *trainer*. Postup môžeme vidieť nižšie:

```
#configure training parameters
args = TrainingArguments(
    output_dir="roberta_only",      #save directory
    learning_rate=2e-5,
    per_device_train_batch_size=16, #train on batches of 16
    per_device_eval_batch_size=32,  #evaluate batches of 32
    num_train_epochs=3,             #3 training cycles
    weight_decay=0.01,

    eval_strategy="epoch",          #evaluate after each epoch
    save_strategy="epoch",          #save after each epoch
    load_best_model_at_end=True,
    metric_for_best_model="f1",     #choose best model based on F1 score

    logging_steps=50,
    fp16=True,
)

#prepare trainer based on previously defined parameters
trainer = Trainer(
    model=model,
    args=args,
    train_dataset=tok["train"],
    eval_dataset=tok["validation"],
    data_collator=collator,
    compute_metrics=compute_metrics,
    processing_class=tokenizer,
)
```

Tréning prebiehal na dátach, ktoré sú rozdelené na tzv. „*batches*“, pričom sme definovali 3 tréningové cykly (epochy). Každý cyklus sme vyhodnotili na datasete *validation*. Po skončení tréningu sme vybrali najlepší model na základe metriky *F1 Score*, pričom sme samotné trénovanie spustili príkazom *trainer.train()*. Výsledné vyhodnotenie trénovania môžeme vidieť na obrázku (*Obrázok 13*).

```
{'test_loss': 0.1030028760433197,
 'test_accuracy': 0.9798464491362764,
 'test_f1': 0.9884551951621771,
 'test_runtime': 2.1997,
 'test_samples_per_second': 473.708,
 'test_steps_per_second': 15.002}
```

Obrázok 13: Výsledné metriky základného modelu RoBERTa.[14]

Vidíme, že *accuracy* a *F1 score* sú pomerne vysoké, čo znamená, že model sa naučil klasifikovať sentiment len na základe samotných textových dát. Ďalej rozšírime pôvodný model o lexikálne *features*, čím vytvoríme hybridný model *RoBERTa*, ktorý okrem samotného textu berie ako vstupné hodnoty aj vopred extrahované lexikálne *features*. Nakoniec porovnáme, či nami vytvorený hybridný model priniesol zlepšenie oproti základnému modelu *RoBERTa*.

4.2.2 Hybridný model RoBERTa

Najskôr sme importovali a konvertovali slovenský lexikón sentimentu na *Pandas DataFrame*. Následne sme vypočítali polaritu, odstránili *sense* čísla a spravili agregáciu všetkých synsetov lexikónu. Ďalej sme zadefinovali výpočet samotných lexikálnych *features* pomocou funkcií *lex_tokens* a *lex_features*, ako môžeme vidieť nižšie:

```
#regex to take only alphabet characters
WORD_RE = re.compile(r"[A-Za-zÁÄČĎÉÍĹĽŇÓÔŔŠŤÚÝŽáäčďéíĺľňóôŕšťúýž]+",
re.UNICODE)

#if text is string, find all matching regex
def lex_tokens(text: str):
    return WORD_RE.findall(text.lower()) if isinstance(text, str) else []

#extract 10 features from text
def lex_features(text: str, strong_thr: float = 0.5):
    toks = lex_tokens(text)
    n = len(toks)

    #safety check
    if n == 0:
        return np.zeros(10, dtype=np.float32)
```

[14] Zdroj Obrázok 13: Autor.

```
#take values from lexicon for each match
vals = [lex_dict[t] for t in toks if t in lex_dict]

#if no match was found, fill with zeroes
if not vals:
    return np.zeros(10, dtype=np.float32)

#assign positive and negative values
vals = np.array(vals, dtype=np.float32)
pos = vals[vals > 0]
neg = vals[vals < 0]

#calculate features
return np.array([
    vals.sum(),                     #polarity sum
    vals.mean(),                    #polarity mean
    len(vals),                      #hits
    len(vals) / n,                  #coverage
    pos.sum() if pos.size else 0,   #sum positive
    neg.sum() if neg.size else 0,   #sum negative
    pos.max() if pos.size else 0,   #max positive
    neg.min() if neg.size else 0,   #max negative
    (pos >= strong_thr).sum(),      #max strong positive
    (neg <= -strong_thr).sum()      #max strong negative
], dtype=np.float32)
```

Aplikovaním funkcie *lex_features* na lematizovaný text datasetov *SlovakSA* sme získali zoznamy číselných hodnôt, ktoré reprezentujú jednotlivé lexikálne *features*. Táto funkcia najskôr nájde všetky zhody textu v lexikóne a následne vypočíta *features*. Ukážku jedného z výsledných datasetov môžeme vidieť na obrázku (*Obrázok 14*).

label	text	lemma_text	lex_feats
1	Bola ústretová,milá a komunikatívna .Šikovná s...	byť ústretový , milý a komunikatívny . šikovný...	[0.6289682388305664, 0.08985260874032974, 7.0,...
1	dobrá obsluha, pekná baba.	dobrý obsluha , pekný baba .	[1.0018938779830933, 0.2504734694957733, 4.0, ...
1	Prijemne vystupovanie, ochota pomôcť	prijemný vystupovanie , ochota pomôcť	[0.1845238208770752, 0.0615079402923584, 3.0, ...
1	Za ochotu	za ochot	[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, ...
1	Ďakujem za rýchle vybavenie a ústretovosť	ďakovať za rýchly vybavenie a ústretovosť	[0.011408727616071701, 0.0028521819040179253, ...
1	Ochotný personál.	ochotný personál .	[0.34375, 0.171875, 2.0, 1.0, 0.34375, 0.0, 0....

Obrázok 14: Ukážka datasetu train_lem s lexikálnymi features.[15]

Ďalej nasleduje konverzia na *DatasetDict* a tokenizácia textu podobne ako pri základnom modely *RoBERTa*. Zadefinovali sme *HybridCollator*, ktorý okrem *paddingu*

[15] Zdroj Obrázok 14: Autor.

vstupu prevedie vstupné lexikálne *features* na tensory. Nižšie môžeme vidieť spomínaný *collator*.

```
#hybrid collator for converting lex_feats to tensors
class HybridCollator:
    def __init__(self, tokenizer):
        self.base = DataCollatorWithPadding(tokenizer)

    def __call__(self, features):
        lex = [f["lex_feats"] for f in features]

        #remove lex_feats before padding
        features2 = []
        for f in features:
            g = dict(f)
            g.pop("lex_feats", None)
            features2.append(g)

        batch = self.base(features2)
        batch["lex_feats"] = torch.tensor(lex, dtype=torch.float32)
        return batch

hybrid_collator = HybridCollator(tokenizer)
```

Nakoniec nám zostáva len samotný hybridný model a jeho trénovanie. Zadefinovali sme triedu *RobertaHybrid* a vytvorili jej inštanciu. Následne sme zadefinovali *hybrid_trainer* a spustili trénovanie.

Po skončení trénovania sme vyhodnotili model na testovacích dátach. Výsledok môžeme vidieť na obrázku (*Obrázok 15*).

```
{'test_loss': 0.1141318753361702,
 'test_accuracy': 0.9779270633397313,
 'test_f1': 0.9873556899395272,
 'test_runtime': 2.0941,
 'test_samples_per_second': 497.592,
 'test_steps_per_second': 15.759}
```

Obrázok 15: Výsledné metriky hybridného modelu RoBERTa.[16]

Na výsledných metrikách pozorujeme, že hybridný model nepredstavuje výrazné zlepšenie oproti základnému modelu *RoBERTa*. Z toho môžeme usúdiť, že transformerové modely ako *RoBERTa* dokážu z textu extrahovať takmer všetky

[16] Zdroj Obrázok 15: Autor.

užitočné informácie, a teda rozšírenie o lexikálnu klasifikáciu neprinieslo do modelu žiadne nové informácie.

Lexikálna klasifikácia sa teda nevyrovná transformerovým modelom, ale môže slúžiť ako základný *baseline* na klasifikáciu sentimentu. Všetky zdrojové kódy spolu s komentármi sú dostupné v prílohe (*Príloha A*).

4.2.3 Lexikálna klasifikácia prostredníctvom Support Vector Machine

S použitím lexikónu sentimentu sme natrénovali *Support Vector* klasifikátor, ktorý sentiment zaraďuje do dvoch tried (*positive*, *negative*). Tento prístup predstavuje výrazné zlepšenie presnosti klasifikácia oproti základnému, ktorý sme aplikovali počas testovania správnosti lexikónu.

V prvom rade sme importovali predpripravený *SlovakSentiWordNet DataFrame*, ktorý sme v predchádzajúcom kroku exportovali pre zjednodušenie, a pripravili sme ho na trénovanie. Najskôr sme vypočítali polaritu, odstránili sense čísla, rozdelíli synsety a nakoniec spravili agregáciu.

Ďalej sme načítali predspracované a lematizované dáta *SlovakSA*, ktoré sme si v predchádzajúcom kroku exportovali spolu s lexikálnymi *features*. Teraz prejdeme na samotný *Support Vector* klasifikátor:

```
X_train = np.array(train_df["lex_feats"].tolist())
y_train = train_df["label"].values

X_val   = np.array(val_df["lex_feats"].tolist())
y_val   = val_df["label"].values

X_test  = np.array(test_df["lex_feats"].tolist())
y_test  = test_df["label"].values

scaler = StandardScaler()

X_train_s = scaler.fit_transform(X_train)
X_val_s   = scaler.transform(X_val)
X_test_s  = scaler.transform(X_test)
```

Najskôr sme si pripravili vstupné dáta do premenných a následne s pomocou *StandardScaler* sme tieto dáta normalizovali. Ďalej sme zadefinovali samotný klasifikátor, ako môžeme vidieť nižšie.

```
svm = LinearSVC(C=1.0, random_state=42)
svm.fit(X_train_s, y_train)
```

Následne sme spustili samotné trénovanie, pričom sme vybrali rôzne hodnoty *C*, ktoré regulujú komplexitu modelu a presnosť klasifikácie. Nakoniec sme z nich vybrali ten model, ktorý má najlepšie výsledky na základe metriky *F1 Score*.

```
Cs = [0.01, 0.1, 1.0, 3.0, 10.0]
best = {"C": None, "f1": -1, "acc": None}

for C in Cs:
    clf = LinearSVC(C=C, random_state=42)
    clf.fit(X_train_s, y_train)
    pred = clf.predict(X_val_s)
    f1 = f1_score(y_val, pred)
    acc = accuracy_score(y_val, pred)
    if f1 > best["f1"]:
        best = {"C": C, "f1": f1, "acc": acc}

best
```

Potom sme model vyhodnotili na testovacích dátach a vypočítali sme výsledné metriky. Výsledok vidíme na obrázku (*Obrázok 16*).

```
TEST Accuracy: 0.8301343570057581
TEST F1: 0.8969132207338381
[[ 95  33]
 [144 770]]
              precision    recall  f1-score   support

           0     0.3975    0.7422    0.5177       128
           1     0.9589    0.8425    0.8969       914

    accuracy                         0.8301      1042
   macro avg     0.6782    0.7923    0.7073      1042
weighted avg     0.8899    0.8301    0.8503      1042
```

Obrázok 16: Výsledné metriky SVM.[17]

Na výsledkoch pozorujeme, že model výrazne preferuje pozitívnu triedu z dôvodu nevyváženosti trénovacích dát *SlovakSA*. SVM model je výrazne lepší ako obyčajná lexikálna klasifikácia, ale pochopiteľne horší ako trasformerový model *RoBERTa*. Tento model sa dá využiť v problémoch klasifikácie, kde výpočtová

[17] Zdroj Obrázok 16: Autor.

nenáročnosť a jednoduchá interpretovateľnosť tréningových váh hrá významnú rolu, napríklad v prípadoch *real-time* klasifikácie nevhodných komentárov alebo recenzii, kde klasifikácia súčasne prebieha na veľkom množstve dát, je nízka výpočtová náročnosť lexikálneho SVM modelu značným prínosom. Interpretovateľnosť výsledkov a trénovacích váh môže byť prínosom pre rôzne domény, kde záleží na extenzívnej analýze správania zákazníkov. Lexikálna analýza SVM modelom môže byť taktiež užitočná pri práci s citlivými dátami, kde je bezpečnosť prioritou.

4.2.4 Automatická klasifikácia textu

Vytvorený *Support Vector* model sme nakoniec upravili tak, aby sme vedeli automaticky klasifikovať akýkoľvek zadaný text. Na tento účel sme pripravili stanza tokenizér a vytvorili funkciu *lemmatize_text*, ako vidíme nižšie.

```
def lemmatize_text(text):
    doc = nlp(text)
    lemmas = []
    for sent in doc.sentences:
        for w in sent.words:
            if w.lemma:
                lemmas.append(w.lemma.lower().replace(" ", "_"))
    return " ".join(lemmas)
```

Ďalej sme vytvorili funkcie *lex_tokens* a *lex_features*, ktoré sme použili na extrahovanie lexikálnych *features* zadaného textu počas behu programu. Tieto funkcie sú tie isté, ktoré sme použili v predošlých častiach práce pri extrakcií lexikálnych *features*, preto ich znova neuvádzame.

Vytvorili sme novú triedu *SlovakSentimentModel*, ktorá slúži na samotnú klasifikáciu zadaného textu. Pozrime sa triedu, ktorá je uvedená nižšie.

```
class SlovakSentimentModel:
    def __init__(self, svm_model, scaler=None):
        self.svm = svm_model
        self.scaler = scaler

    def predict(self, text):
      #lemmatize text and extract features
      lem_text = lemmatize_text(text)
      feats = lex_features(lem_text)
      X = feats.reshape(1, -1)
      X = self.scaler.transform(X)
```

```
#predict sentiment and assign pos/neg value
pred = int(self.svm.predict(X)[0])
margin = float(self.svm.decision_function(X)[0])

label = "positive" if pred == 1 else "negative"

return {
    "label": label,
    "class": pred,
    "confidence": round(margin, 3)
}
```

Funkcia *predict* predstavuje hlavnú logiku klasifikácie textu. Najskôr lematizuje text a extrahuje *features*, a následne normalizuje vstup prostredníctvom *StandartScaler*. Nakoniec vyhodnotí výsledný sentiment s použitím natrénovaného *Support Vector* klasifikátora. Použitie triedy *SlovakSentimentModel* vidíme na obrázku (*Obrázok 17*).

```
model = SlovakSentimentModel(svm, scaler)

print(model.predict("Milujem tento film."))
print(model.predict("Nenávidím tento film."))

{'label': 'positive', 'class': 1, 'confidence': 1.155}
{'label': 'negative', 'class': 0, 'confidence': -0.966}
```

Obrázok 17: Príklad použitia triedy SlovakSentimentModel.[18]

Vidíme, že model správne vyhodnotil triedu zadaného textu, pričom vidíme aj hodnotu *confidence*, ktorá značí s akou mierou si je model istý danou klasifikáciou.

4.3 TESTOVANIE

V praktickej časti práce sme vytvorili slovenský lexikón sentimentu na základe *SentiWordNet 3.0*, a následne sme otestovali a porovnali niekoľko prístupov klasifikácie sentimentu. Najskôr sme testovali spoľahlivosť samotného lexikónu a prišli sme k záveru, že vytvorený lexikón má zachovanú výpovednú hodnotu aj po preklade synsetov SWN. Robili sme porovnanie na dátach *SlovakSA*, pričom sme porovnali dáta s lematizáciou a bez lematizácie. Po lematizácií dát sme spozorovali výrazne viac nájdených zhôd v lexikóne, keďže lexikón obsahuje jednotlivé slová v základnom tvare.

[18] Zdroj Obrázok 17: Autor.

Ďalej sme porovnávali základný model *RoBERTa* a hybridný model, ktorý predstavuje rozšírenie základného modelu *RoBERTa* o lexikálne *features*, ktoré sme získali z nášho lexikónu. Zistili sme, že hybridný model nepredstavuje zlepšenie oproti základnému modelu *RoBERTa*, keďže transformerové modely dokážu extrahovať takmer všetky informácie zo vstupného textu. Lexikálne *features* teda nepridali žiadnu novú informáciu do modelu.

Nakoniec sme vytvorili lineárny *Support Vector Machine*, ktorý má najlepšie výsledky zo všetkých nami testovaných lexikálnych prístupov. Náš SVM model výrazne preferuje pozitívnu triedu, pretože trénovacie dáta sú dosť nevyvážené. SVM model sa síce nevyrovná transformerovým modelom, ale má niekoľko výhod, napríklad:

- transparentnosť,
- nízka výpočtová náročnosť,
- adaptabilita.

V tabuľke (*Tabuľka 2*) vidíme porovnanie všetkých spomínaných prístupov klasifikácie sentimentu.

Tabuľka 2: Porovnanie prístupov klasifikácie sentimentu.[19]

Model	**Accuracy**	**Precision**	**Recall**	**F1**	**Poznámka**
Lexikálna klasifikácia (bez lematizácie)	0.594	0.922	0.570	0.705	nízke pokrytie lexikónu, veľa False Negative
Lexikálna klasifikácia (s lematizáciou)	0.700	0.917	0.712	0.802	výrazné zlepšenie pokrytia lexikónu
RoBERTa (baseline)	**0.980**	–	–	**0.988**	najlepší model
RoBERTa + Lexikón (hybrid)	0.978	–	–	0.987	približne rovnaký ako baseline
Lexikón + Linear SVM	0.830	0.959	0.843	0.897	najlepší lexikálny prístup

[19] Zdroj Tabuľka 2: Autor.

4.4 DISKUSIA

Existujú rôzne prístupy analýzy sentimentu, ktoré môžeme zaradiť do štyroch kategórií: lexikálna, strojové učenie, hybridná a transformerová. Lexikálny prístup je výrazne špecifický pre jednotlivé domény, pretože významy jednotlivých termínov sa môžu naprieč doménami líšiť.

Metódy strojového učenia ako *Naive Bayes* alebo *Support Vector Machine* sa dokážu naučiť užitočné informácie z trénovacích dát. Konkrétne *Support Vector Machine* má silnú teoretickú rovinu a vo veľa prípadoch dosahuje lepšie výsledky ako iné metódy strojového učenia (Mao et al., 2024).

Hybridný prístup predstavuje kombináciu tradičných metód strojového učenia a lexikálnych metód, pričom kombinuje lingvistickú analýzu a kontext slov. Konkrétne autori (Chang et al., 2020) využili hybridnú metódu na kombináciu *SVM* a reliéfových algoritmov. Na trénovanie použili takmer 7000 *tweet*ov, pričom ich model prekonáva väčšinu ostatných modelov s presnosťou okolo 80 až 85% v závislosti od dát.

Transformery používajú takzvaný *self-attention* mechanizmus na modelovanie, čo znamená, že pri neobmedzenej výpočtovej kapacite dokážu transformery teoreticky analyzovať neobmedzené množstvo textu. V praxi to znamená, že tieto modely sú výrazne obmedzené pamäťou a výpočtovou kapacitou.

Transformerové modely ako *RoBERTa* predstavujú výrazné zlepšenie oproti lexikálnym prístupom, a to vďaka hĺbkovej analýze kontextu vstupných dát. Autori (Tran et al., 2025) vytvorili model *CombViSA*, ktorý vznikol kombináciou transformerového modelu *PhoBERT V2* s lexikálnymi *features* extrahovanými z vietnamského *SentiWordNet*-u. Tento model predstavuje výrazné zlepšenie oproti základnému modelu *PhoBERT*, ktorý sám predstavuje zlepšenie oproti základnému vietnamskému *BERT* modelu.

Autori (Nasiopoulos et al., 2025) porovnávali rôzne metódy analýzy sentimentu v doméne financií, pričom pozorujú, že predtrénované LLM modely ako *GPT-4o* predstavujú približne 8% zlepšenie oproti modelom *BERT* a výrazné zlepšenie oproti tradičným metódam strojového učenia, akým je *SVM* a lexikálna analýza.

Porovnaním jednotlivých prístupov analýzy sentimentu môžeme usúdiť, že transformerové predtrénované modely predstavujú zatiaľ najlepšie riešenie problémov klasifikácie sentimentu. Podobné výsledky pozorujeme v tejto práci, kde základný model *RoBERTa* predstavuje najvýkonnejší spôsob klasifikácie sentimentu. Avšak nami

prezentovaný *SVM* model založený na vytvorenom lexikóne *SlovakSentiWordNet*, dosahuje veľmi dobré výsledky a má niekoľko výhod oproti transformerovým modelom v konkrétnych prípadoch.

Extenzívnym testovaním sme preukázali, že nami vytvorený lexikón *SlovakSentiWordNet* predstavuje spoľahlivý zdroj pre lexikálne prístupy analýzy sentimentu.

Výkon nášho *SVM* modelu je možné výrazne zlepšiť použitím rozsiahlejších trénovacích dát, keďže dáta *SlovakSA*, ktoré sme použili na trénovanie modelu výrazne preferujú pozitívnu klasifikáciu sentimentu a obsahujú relatívne malé množstvo ohodnotených recenzií.

ZÁVER

Získané výsledky v praktickej časti tejto záverečnej práci sú v súlade s vopred stanovenými cieľmi a predpokladmi. V práci sme sa podrobne venovali rôznym prístupom lexikálnej analýzy sentimentu, pričom sme otestovali niekoľko metód.

Vytvorili sme slovenský lexikón sentimentu s názvom *SlovakSentiWordNet* (*Príloha A2*). Tento lexikón sme vytvorili kombináciou synsetov litovského *WordNet*-u v slovenskom jazyku a strojovým prekladom synsetov anglického SentiWordNet-u 3.0, pričom sme zachovali pôvodné hodnoty polarity sentimentu. Pri vytváraní slovníku sme narazili na niekoľko problémov, ktoré sme museli vyriešiť. Rovnako ako aj *SentiWordNet*, slovenský litovský *WordNet* je založený na *Princeton WordNet*-e, ale odlišujú sa formátovaním, ktoré sa nám úspešne podarilo zjednotiť. Litovský *WordNet* taktiež neobsahuje významové číslovanie synsetov pôvodného *WordNet*-u, ktoré sme museli doplniť. Synsety *SentiWordNet*-u sme po častiach preložili s použitím *HelsinkiNLP*, ktorý podporuje aj slovenský jazyk. Zlúčením týchto dvoch zdrojov sme dostali lexikón *SlovakSentiWordNet*, ktorý obsahuje všetky synsety pôvodného *SentiWordNet*-u v slovenskom jazyku.

Výsledný lexikón sme testovali na dátach *SlovakSA*, ktoré obsahujú recenzie v slovenskom jazyku ohodnotené polaritou sentimentu. Testovali sme na dátach bez lematizácie a s lematizáciou, pričom dáta s lematizáciou priniesli výrazne lepšie výsledky. Testovaním sme preukázali, že vytvorený lexikón *SlovakSentiWordNet* je spoľahlivý zdroj pre lexikálnu analýzu sentimentu.

V ďalšej časti práce sme vytvorili hybridný model *RoBERTa*, ktorý sa od základného modelu líši pridaním lexikálnych vlastností na vstupe, ktoré boli extrahované z lexikónu *SlovakSentiWordNet*. Tento model sme porovnávali so základným modelom *RoBERTa*, pričom sme prišli k záveru, že hybridný model nepriniesol zlepšenie oproti základnému modelu *RoBERTa*. Tento výsledok je pochopiteľný, keďže transformerové modely ako *RoBERTa* dokážu extrahovať takmer všetky informácie zo samotného vstupného textu. Podobne ako pri testovaní lexikónu, spomínané modely *RoBERTa* boli trénované a testované na dátach *SlovakSA*.

Nakoniec sme vytvorili *Support Vector Machine* založený na lexikálnych údajoch extrahovaných z nášho lexikónu, ktorý dokáže klasifikovať text do dvoch tried, a to: pozitívny a negatívny. Vytvorený *SVM* model predstavuje najlepší nami testovaný

lexikálny prístup. Vzhľadom na pozitívnu orientáciu dát *SlovakSA*, náš *SVM* model preferuje pozitívnu triedu. Tento nedostatok je možné odstrániť dodatočným trénovaním na rôznorodých dátach v slovenskom jazyku.

Nami dosiahnuté výsledky predstavujú dobrý základný *baseline* pre analýzu sentimentu v slovenskom jazyku. Vytvorený *SVM* model je pochopiteľne menej výkonný ako trasformerové modely ako *RoBERTa*, ale má oproti nim niekoľko výhod, ako interpretovateľnosť a nízka výpočtová náročnosť.

ZOZNAM BIBLIOGRAFICKÝCH ODKAZOV

Ahmad, M., Aftab, S., Muhammad, S. S., & Ahmad, S. (2017). Machine Learning Techniques for Sentiment Analysis: A Review. *INTERNATIONAL JOURNAL OF MULTIDISCIPLINARY SCIENCES AND ENGINEERING*, *8*(3). www.ijmse.org

Almeida, F., & Xexéo, G. (2019). *Word Embeddings: A Survey*. https://arxiv.org/pdf/1901.09069

Amazon Comprehend – Features. (2025). https://aws.amazon.com/comprehend/features/?utm_source=chatgpt.com

Baccianella, S., Esuli, A., & Sebastiani, F. (2010). *SENTIWORDNET 3.0: An Enhanced Lexical Resource for Sentiment Analysis and Opinion Mining*. http://wordnetcode.princeton.

Badman, A., & Kosinski, M. (2024). *What is Data? | IBM*. https://www.ibm.com/think/topics/data

Bonta, V., Kumaresh, N., & Janardhan, N. (2019). A Comprehensive Study on Lexicon Based Approaches for Sentiment Analysis. *Asian Journal of Computer Science and Technology*, *8*(S2), 1–6. https://doi.org/10.51983/AJCST-2019.8.S2.2037

Cao, H., & SAS France, A. (2024). *Recent advances in text embedding: A Comprehensive Review of Top-Performing Methods on the MTEB Benchmark*. https://arxiv.org/pdf/2406.01607v2

Carvalho, A., Levitt, A., Levitt, S., Khaddam, E., Benamati, J., Carvalho, A., Levitt, A., Khaddam, S., & Benamati, E. (2019). *Off-The-Shelf Artificial Intelligence Technologies for Sentiment and Emotion Analysis: A Tutorial on Using IBM Natural Language Processing*. https://doi.org/10.17705/1CAIS.04443

Chang, J. R., Liang, H. Y., Chen, L. S., & Chang, C. W. (2020). Novel feature selection approaches for improving the performance of sentiment classification. *Journal of Ambient Intelligence and Humanized Computing 2020*, 1–14. https://doi.org/10.1007/s12652-020-02468-z

Cloud Natural Language API | Google Cloud. (2025). https://cloud.google.com/natural-language/docs/analyzing-sentiment?utm_source=chatgpt.com

Continuous bag of words (CBOW) in NLP - GeeksforGeeks. (2025). https://www.geeksforgeeks.org/nlp/continuous-bag-of-words-cbow-in-nlp/

Das Debasish. (2025, November). *Self-Attention: The Simple Mechanism That Made ChatGPT Possible*. https://pub.towardsai.net/transformer-architecture-self-attention-3f947540871d

Detect sentiment | IBM Cloud Docs. (n.d.). Retrieved October 25, 2025, from https://cloud.ibm.com/docs/discovery-data?topic=discovery-data-nlu-sentiment

DetectSentiment - Amazon Comprehend API. (n.d.). Retrieved October 17, 2025, from https://docs.aws.amazon.com/comprehend/latest/APIReference/API_DetectSentiment.html?utm_source=chatgpt.com

Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *NAACL HLT 2019 - 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies - Proceedings of the Conference*, *1*, 4171–4186. https://arxiv.org/pdf/1810.04805

Egger, R. (2022). Text Representations and Word Embeddings: Vectorizing Textual Data. *Tourism on the Verge*, *Part F1051*, 335–361. https://doi.org/10.1007/978-3-030-88389-8_16

Esuli, A., & Sebastiani, F. (2006). *SENTIWORDNET: A Publicly Available Lexical Resource for Opinion Mining*. http://www-2.cs.cmu.edu/

FastText Embedding technique. - Medium. (2014). https://medium.com/@abhishekjainindore24/fasttext-embedding-technique-9b5d1fba4b50

Ghannay, S., Favre, B., Estève, Y., & Camelin, N. (2016). *Word Embedding Evaluation and Combination* (pp. 300–305). https://aclanthology.org/L16-1046/

Glove Word Embedding in NLP - GeeksforGeeks. (2015). https://www.geeksforgeeks.org/nlp/Glove-Word-Embedding-in-NLP/

Grzegorczyk Advisor, K., Witold Dzwinel, P., Co-advisor, Ds., & Kurdziel, M. (2019). *Vector representations of text data in deep learning*.

Jacob Eisenstein. (2019). *Introduction to Natural Language Processing - Jacob Eisenstein - Google Books*. https://books.google.sk/books?hl=en&lr=&id=72yuDwAAQBAJ&oi=fnd&pg=PR5&dq=natural+language+processing+introduction&ots=gX9O_-5gn_&sig=y9fCGPJgZhIsdn764kGTkbxG2Ec&redir_esc=y#v=onepage&q=natural%20language%20processing%20introduction&f=false

Lee, D. L., & Kong, H. (1997). *Document Ranking and the Vector-Space Model*.

Lin, T., Wang, Y., Liu, X., & Qiu, X. (2022). A survey of transformers. *AI Open*, *3*, 111–132. https://doi.org/10.1016/J.AIOPEN.2022.10.001

Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., Stoyanov, V., & Allen, P. G. (2019). *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. https://arxiv.org/pdf/1907.11692

Mao, Y., Liu, Q., & Zhang, Y. (2024). Sentiment analysis methods, applications, and challenges: A systematic literature review. *Journal of King Saud University - Computer and Information Sciences*, *36*(4), 102048. https://doi.org/10.1016/j.jksuci.2024.102048

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). *Distributed Representations of Words and Phrases and their Compositionality*.

Nasiopoulos, D. K., Roumeliotis, K. I., Sakas, D. P., Toudas, K., & Reklitis, P. (2025). Financial Sentiment Analysis and Classification: A Comparative Study of Fine-Tuned Deep Learning Models. *International Journal of Financial Studies 2025, Vol. 13, Page 75*, *13*(2), 75. https://doi.org/10.3390/ijfs13020075

NLP Cloud | LangChain. (2025). https://python.langchain.com/docs/integrations/llms/nlpcloud/

Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global vectors for word representation. *EMNLP 2014 - 2014 Conference on Empirical Methods in Natural Language Processing, Proceedings of the Conference*, 1532–1543. https://doi.org/10.3115/V1/D14-1162

POS(Parts-Of-Speech) Tagging in NLP. (2025). https://www.geeksforgeeks.org/nlp/nlp-part-of-speech-default-tagging/

Qader, W. A., Ameen, M. M., & Ahmed, B. I. (2019). An Overview of Bag of Words;Importance, Implementation, Applications, and Challenges. *Proceedings of the 5th International Engineering Conference, IEC 2019*, 200–204. https://doi.org/10.1109/IEC47844.2019.8950616

Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). *DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter*. https://arxiv.org/pdf/1910.01108

Sharma, H. D., & Goyal, P. (2023). An Analysis of Sentiment: Methods, Applications, and Challenges. *Engineering Proceedings 2023, Vol. 59, Page 68*, *59*(1), 68. https://doi.org/10.3390/ENGPROC2023059068

Speer, R., & Chin, J. (2019). *An Ensemble Method to Produce High-Quality Word Embeddings*. http://commoncrawl.org/

Supriyono, Wibawa, A. P., Suyono, & Kurniawan, F. (2024). Advancements in natural language processing: Implications, challenges, and future directions. *Telematics and Informatics Reports*, *16*, 100173. https://doi.org/10.1016/J.TELER.2024.100173

Suri, M. (2022). *A Dummy's Guide to Word2Vec.* https://medium.com/@manansuri/a-dummys-guide-to-word2vec-456444f3c673

Taboada, M., Brooke, J., Tofiloski, M., Voll, K., & Stede, M. (2011). *Lexicon-Based Methods for Sentiment Analysis*. http://direct.mit.edu/coli/article-pdf/37/2/267/1798865/coli_a_00049.pdf

Tran, H.-V., Bui, V.-T., & Tran, L.-Q. (2025). *Expanding Vietnamese SentiWordNet to Improve Performance of Vietnamese Sentiment Analysis Models*. http://arxiv.org/abs/2501.08758

Turney, P. D. (2002). *Thumbs Up or Thumbs Down? Semantic Orientation Applied to Unsupervised Classification of Reviews*. http://www.google.com

Vaswani, A., Brain, G., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2023). *Attention Is All You Need*.

Vechtomova, O. (2009). Introduction to Information RetrievalChristopher D. Manning, Prabhakar Raghavan, and Hinrich Sch. *Computational Linguistics*, *35*(2), 307–309. https://doi.org/10.1162/coli.2009.35.2.307

Webster, J. J., & Kit, C. (1992). *TOKENIZATION AS THE INITIAL PHASE IN NLP*.

What are stop words in NLP? (n.d.). Retrieved November 14, 2025, from https://milvus.io/ai-quick-reference/what-are-stop-words-in-nlp

ZOZNAM PRÍLOH

Príloha A – Odkaz na repozitár práce

PRÍLOHA A

Príloha A

- A1 Odkaz na Repozitár GIT: *https://github.com/MatejLuzak/Lexikalna-Analyza-Sentimentu-Aplikovatelna-na-Slovenske-texty/*
- A2 Odkaz na Slovenský *SentiWordNet*: *https://github.com/MatejLuzak/Lexikalna-Analyza-Sentimentu-Aplikovatelna-na-Slovenske-texty/blob/main/Output/SentiWordNet_3.0.0_SK.txt*
- A3 Odkaz na kód: *https://github.com/MatejLuzak/Lexikalna-Analyza-Sentimentu-Aplikovatelna-na-Slovenske-texty/tree/main/Output/Notebook*
- A4 Odkaz na čiastkové *CSV* súbory: *https://github.com/MatejLuzak/Lexikalna-Analyza-Sentimentu-Aplikovatelna-na-Slovenske-texty/tree/main/Output/CSV*